Physics-Informed Neural Networks

Scientific Machine Learning and Simulation Intelligence

Francesca Cairoli

University of Trieste, 2026/27

Problem

More often than not, in the course of analyzing complex physical, biological or engineering systems, the **cost of data acquisition is prohibitive**, and we are inevitably faced with the challenge of drawing conclusions and **making decisions under partial information**.

In this **small data regime**, the vast majority of state-of-the-art machine learning techniques (e.g., deep/convolutional/recurrent neural networks):

- lack robustness
- fail to provide any guarantees of convergence.

Problem

Deep learning models try to accurately identify a nonlinear map from a few – potentially very high-dimensional – input and output data pairs.

For many cases pertaining to the modeling of physical and biological systems, there exists a **vast amount of prior knowledge** that is currently **not being utilized** in modern machine learning practice.

This **prior information** can act **as a regularization** agent that constrains the space of admissible solutions to a manageable size.

Encoding such structured information into a learning algorithm results in **amplifying the information content of the data** that the algorithm sees, enabling it to quickly steer itself towards the right solution and generalize well even when only a few training examples are available.

Introduction to PINN

Neural networks that are trained to solve supervised learning tasks while respecting any given law of physics described by general nonlinear partial differential equations.

Two **classes of problems**:

1. data-driven solution
2. data-driven identification

of partial differential equations.

Two **types of algorithms**:

1. continuous time models
2. discrete time models.

Raissi, Maziar, Paris Perdikaris, and George E. Karniadakis. "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations." *Journal of Computational physics* 378 (2019)

Problem Statement

Consider **parametrised and nonlinear partial differential equation** of the general form

$$u_t + \mathcal{N}[u;\lambda] = 0,\ x \in \Omega,\ t \in [0,T]$$

where

$u(t,x)$ denotes the latent solution,

$\mathcal{N}[\,\cdot\,;\lambda]$ is a nonlinear operator parametrised by λ, and

Ω is a subset of $\mathbb{R}^D$.

Remark: this setup encapsulates a wide range of problems in mathematical physics including conservation laws, diffusion processes, advection–diffusion–reaction systems, and kinetic equations.

This formulation assuming no forcing terms $f(t,x)$

Example

Burgers Equation

Application domains:

- Fluid mechanics
- Nonlinear acoustic
- Gas dynamics
- Traffic flow

convection-diffusion equation

PDE: $u_t = \lambda_2 u_{xx} - \lambda_1 u u_x$

diffusion coeffient

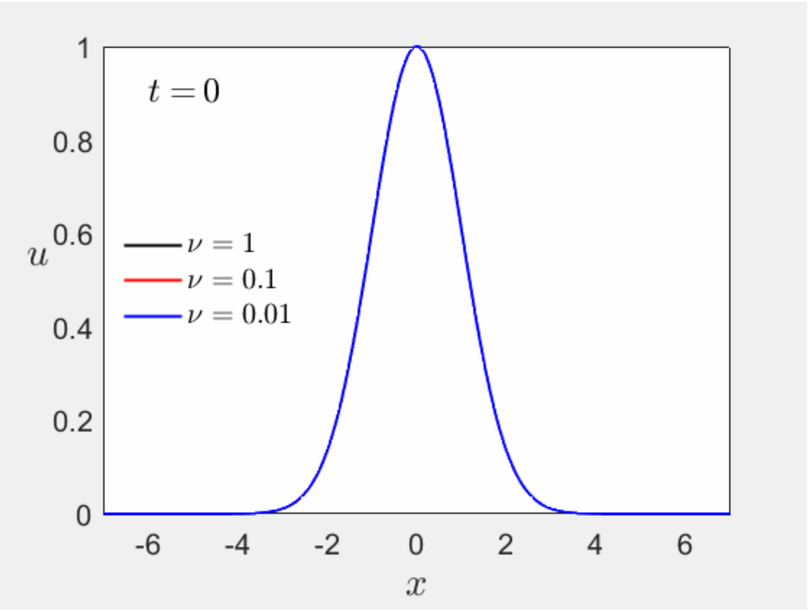
Parameter $\lambda = (\lambda_1, \lambda_2)$

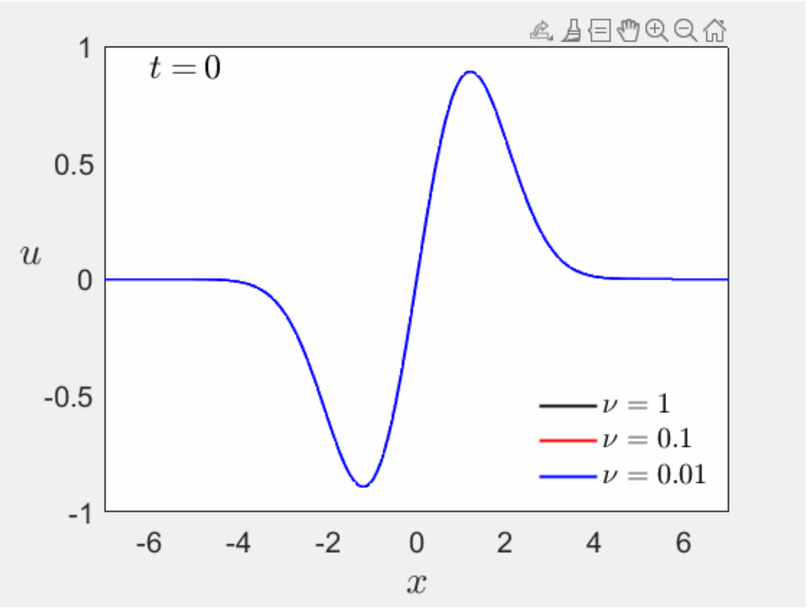
Nonlinear operator $\mathcal{N}[u;\lambda] = \lambda_1 u u_x - \lambda_2 u_{xx}$

Solution: $u(t,x)$

Solutions of the Burgers equation starting from a Gaussian initial condition $u(0,x) = e^{-x^2/2}$

$\nu = \lambda_2$





N-wave type solution of the Burger equation starting from the initial condition $u(0,x) = e^{-(x-1)^2/2} - e^{-(x+1)^2/2}$

Consider **parametrised and nonlinear partial differential equation** of the general form

$$u_t + \mathcal{N}[u;\lambda] = 0,\ x \in \Omega,\ t \in [0,T]$$

where

$u(t,x)$ denotes the latent solution,

$\mathcal{N}[\,\cdot\,;\lambda]$ is a nonlinear operator parametrised by λ, and

Ω is a subset of $\mathbb{R}^D$.

Given noisy measurements of the system, **solution of two distinct problems**:

1. INFERENCE (data-driven solutions of PDE):

   Given fixed model parameters λ, what can be said about the unknown hidden state $u(t,x)$ of the system?

2. LEARNING (data-driven identification of PDE):

   What are the parameters λ that best describe the observed data?

1. Data-driven solutions of PDEs

Problem of computing data-driven solutions to partial differential equations of the general form

$$u_t + \mathcal{N}[u] = 0,\ x \in \Omega,\ t \in [0,T].$$

λ fixed
Initial conditions over t
Boundary conditions over x

a. Continuous time models

Define $f(t,x) := u_t + \mathcal{N}[u]$ and approximate $u(t,x)$ by a deep neural network.

$f(t,x)$ → physics-informed neural network that can be derived using chain rule for differentiating compositions of functions using automatic differentiation and has the same parameters as the network representing $u(t,x)$, albeit with different activation functions due to the action of the differential operator $\mathcal{N}$.

$\mathcal{N}$ often contains derivatives of $u(t,x)$ wrt x

1.a. Data-driven solutions of PDEs

The shared parameters between the neural networks $u(t, x)$ and $f(t, x)$ can be learned by minimizing the mean squared error loss:

$$MSE = MSE_u + MSE_f$$

where

$$MSE_u = \frac{1}{N_u} \sum_{i=1}^{N_u} | u(t_u^i, x_u^i) - u^i |^2,$$

and

$$MSE_f = \frac{1}{N_f} \sum_{i=1}^{N_f} | f(t_f^i, x_f^i) |^2.$$

Here $\{t_u^i, x_u^i, u^i\}_{i=1}^{N_u}$ denote the initial and boundary training data on $u(t, x)$ and $\{t_f^i, x_f^i\}_{i=1}^{N_f}$ specify the collocation points for $f(t, x)$.

1.a. Data-driven solutions of PDEs

The shared parameters between the neural networks $u(t,x)$ and $f(t,x)$ can be learned by minimizing the mean squared error **loss**:

$$MSE = MSE_u + MSE_f$$

where

$$MSE_u = \frac{1}{N_u}\sum_{i=1}^{N_u} |\, u(t_u^i, x_u^i) - u^i \,|^2,$$

corresponds to initial and boundary data

and

$$MSE_f = \frac{1}{N_f}\sum_{i=1}^{N_f} |\, f(t_f^i, x_f^i) \,|^2.$$

enforces the structure imposed by the PDE at a finite set of locations points

Here $\{t_u^i, x_u^i, u^i\}_{i=1}^{N_u}$ denote the initial and boundary training data on $u(t,x)$ and $\{t_f^i, x_f^i\}_{i=1}^{N_f}$ specify the collocation points for $f(t,x)$.

Open the black-box

Automatic differentiation in general, and the back-propagation algorithm in particular, is currently the dominant approach for training deep models by taking their derivatives with respect to the parameters (e.g., weights and biases) of the models. Here, we use the exact same automatic differentiation techniques to physics-inform neural networks by taking their derivatives with respect to their input coordinates (i.e., space and time) where the physics is described by partial differential equations.

This approach acts as a **regularisation mechanism** that allows us to **use relatively simple feed-forward neural network architectures and train them with small amounts of data**.

Soft constraints

Some tricks to force hard constraints (such as conservation laws)

Example

Schrödinger Equation

This example aims to highlight the ability of PINN to handle periodic boundary conditions, complex-valued solutions, as well as different types of nonlinearities in the governing partial differential equations.

The *one-dimensional nonlinear Schrödinger equation* is a classical field equation that is used to study quantum mechanical systems

$$ih_t + 0.5h_{xx} + |h|^2 h = 0 \qquad x \in [-5,5],\ t \in [0,\pi/2]$$

$$h(0,\ x) = 2\ \text{sech}(x) \longrightarrow \text{initial conditions}$$

$$h(t,-5) = h(t,5)$$

$$h_x(t,-5) = h_x(t,5) \longrightarrow \text{periodic boundary conditions}$$

where $h(t,x)$ is the complex-valued solution.

Let $f := ih_t + 0.5h_{xx} + |h|^2 h$ and proceed by placing a complex-valued neural network prior on $h(t,x)$, i.e. we place a multi-out neural network prior on $h(t,x) = [u(t,x)\ \ v(t,x)]$, where u denotes the real part of h and v is the imaginary part.

Example

Schrödinger Equation

The shared parameters of the neural networks $h(t,x)$ and $f(t,x)$ can be learned by minimizing the mean squared error loss

$$MSE = MSE_0 + MSE_b + MSE_f$$

where

$$MSE_0 = \frac{1}{N_0}\sum_{i=1}^{N_0} |\, h(0,x_0^i) - h_0^i \,|^2,$$

$$MSE_b = \frac{1}{N_b}\sum_{i=1}^{N_b} \left(|\, h(t_b^i,-5) - h(t_b^i,5) \,|^2 + |\, h_x(t_b^i,-5) - h_x(t_b^i,5) \,|^2 \right),$$

and

$$MSE_f = \frac{1}{N_f}\sum_{i=1}^{N_f} |\, f(t_f^i, x_f^i) \,|^2.$$

$\{x_0^i, h_0^i\}_{i=1}^{N_0}$ initial data

$\{t_b^i\}_{i=1}^{N_b}$ collocation points on the boundary

$\{t_f^i, x_f^i\}_{i=1}^{N_f}$ collocation points on $f(t,x)$

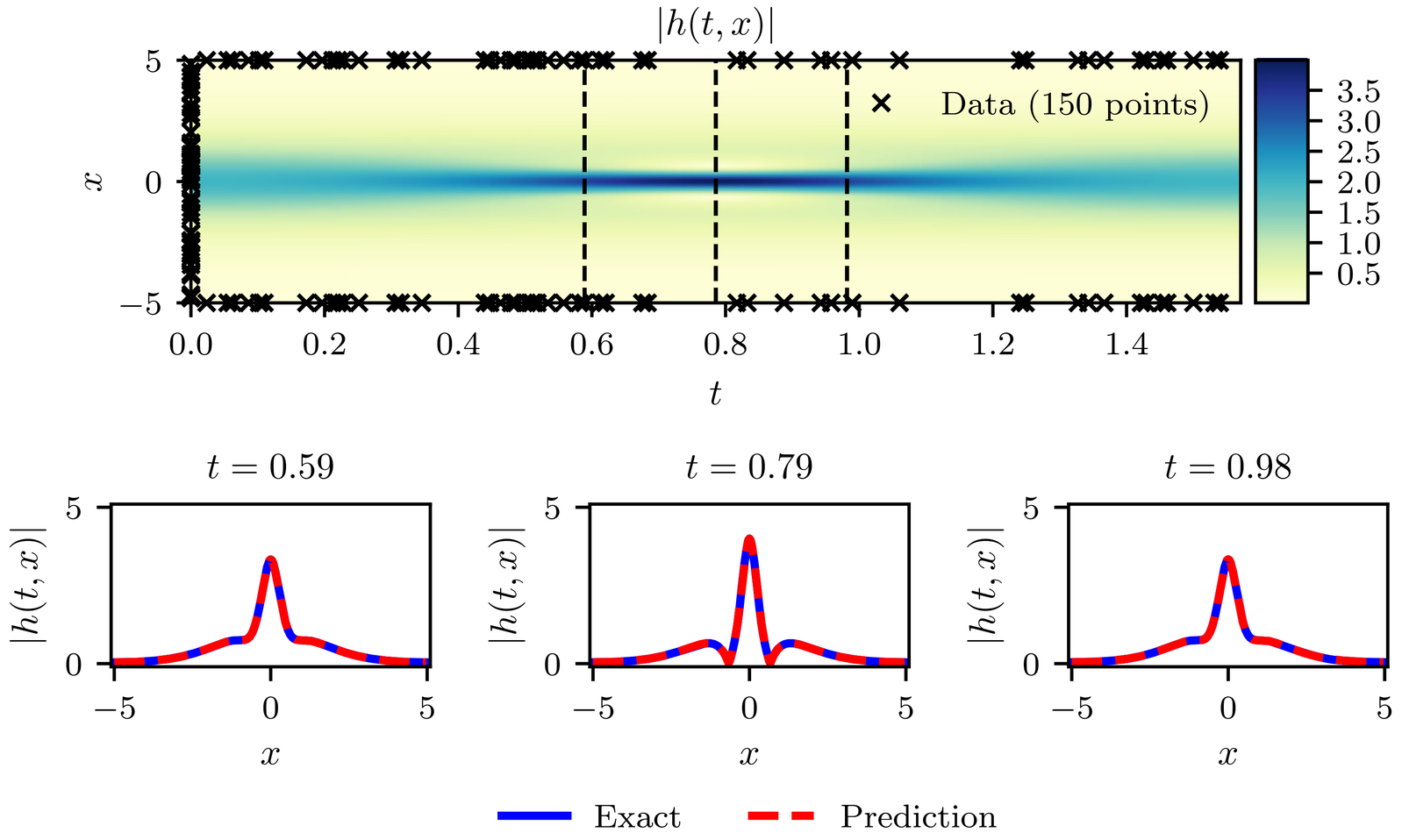


Fig. 1. *Schrodinger equation: Top:* Predicted solution $|h(t,x)|$ along with the initial and boundary training data. In addition we are using 20,000 collocation points generated using a Latin Hypercube Sampling strategy. *Bottom:* Comparison of the predicted and exact solutions corresponding to the three temporal snapshots depicted by the dashed vertical lines in the top panel. The relative $\mathbb{L}_2$ error for this case is $1.97 \cdot 10^{-3}$.

1.a. Demo session

https://github.com/maziarraissi/PINNs

https://github.com/maziarraissi/PINNs/tree/master/main/continuous_time_inference

1.a. Bottleneck

One potential **limitation of the continuous time** neural network models considered so far stems from the need to use a **large number of collocation points** N_f in order to enforce physics-informed constraints in the entire spatio-temporal domain.

Although this poses no significant issues for problems in one or two spatial dimensions, it may introduce a severe **bottleneck in higher dimensional problems**, as the total number of collocation points needed to globally enforce a physics-informed constrain (i.e., in our case a partial differential equation) will increase exponentially.

We now see a different approach that circumvents the need for collocation points by introducing a more structured neural network representation leveraging the classical Runge–Kutta time-stepping schemes.

1. Data-driven solutions of PDEs

λ fixed
Initial conditions over t
Boundary conditions over x

b. Discrete time models

Δt = time step

$$u_t + \mathcal{N}[u] = 0,\ x \in \Omega,\ t \in [0,T]$$

Let's apply the general form of Range-Kutta methods with q stages to the PDE and obtain

$$u^{n+c_i} = u^n - \Delta t \sum_{j=1}^{q} a_{ij} \mathcal{N}[u^{n+c_j}], \qquad i = 1,\ldots,q$$

$$u^{n+1} = u^n \ominus \Delta t \sum_{j=1}^{q} b_j \mathcal{N}[u^{n+c_j}]$$

Minus comes from the fact that: $u_t = -\mathcal{N}[u]$

Here, $u^{n+c_j}(x) := u(t^n + c_j \Delta t, x)$ for $j = 1,\ldots,q$. This general form encapsulates both implicit and explicit time-stepping schemes, depending on the choice of the parameters $\{a_{ij}, b_j, c_j\}$.

We proceed by placing a **multi-output neural network** prior on $[u^{n+c_1}(x), \ldots, u^{n+c_q}(x), u^{n+1}(x)]$:

$$NN_\theta(x) = [u_\theta^{n+c_1}(x), \ldots, u_\theta^{n+c_q}(x), u_\theta^{n+1}(x)]$$

(for a generic s)

0					
c_2	a_{21}				
c_3	a_{31}	a_{32}			
$\vdots$	$\vdots$		$\ddots$		
c_s	a_{s1}	a_{s2}	$\cdots$	$a_{s,s-1}$	
	b_1	b_2	$\cdots$	b_{s-1}	b_s

b_i are the *weights* and c_i are the *nodes*

Consistency conditions:

$$\sum_{i=1}^{s} b_i = 1$$

$$\sum_{j=1}^{i-1} a_{ij} = c_i$$

$$K_1 = \mathcal{N}[u_n],$$

$$K_2 = \mathcal{N}\left[u_n - \frac{\Delta t}{2} K_1\right], \qquad u^{n+c_2}$$

$$K_3 = \mathcal{N}\left[u_n - \frac{\Delta t}{2} K_2\right],$$

$$K_4 = \mathcal{N}\left[u_n - \Delta t\, K_3\right],$$

$$u_{n+1} = u_n - \Delta t \left(\frac{1}{6}K_1 + \frac{1}{3}K_2 + \frac{1}{3}K_3 + \frac{1}{6}K_4\right).$$

0				
1/2	1/2			
1/2	0	1/2		
1	0	0	1	
	1/6	1/3	1/3	1/6

Runge Kutta 4-th order

$$u^{n+c_i} = u^n - \Delta t \sum_{j=1}^{q} a_{ij} \mathcal{N}[u^{n+c_j}], \qquad i = 1, \ldots, q$$

$$u^{n+1} = u^n - \Delta t \sum_{j=1}^{q} b_j \mathcal{N}[u^{n+c_j}]$$

1. Data-driven solutions of PDEs

b. Discrete time models

Explicit Butcher tableau

(for a generic s)

$$\begin{array}{c|cccc} 0 & & & & \\ c_2 & a_{21} & & & \\ c_3 & a_{31} & a_{32} & & \\ \vdots & \vdots & & \ddots & \\ c_s & a_{s1} & a_{s2} & \cdots & a_{s,s-1} \\ \hline & b_1 & b_2 & \cdots & b_{s-1} \quad b_s \end{array}$$

b_i are the *weights* and c_i are the *nodes*

Consistency conditions:

$$\sum_{i=1}^{s} b_i = 1$$

$$\sum_{j=1}^{i-1} a_{ij} = c_i$$

$$\begin{array}{c|c} 0 & \\ \hline & 1 \end{array}$$

Forward Euler

$$\begin{array}{c|cc} 0 & & \\ 1/2 & 1/2 & \\ \hline & 0 & 1 \end{array}$$

Explicit midpoint (RK2)
($\alpha = 1$)

$$\begin{array}{c|cc} 0 & & \\ \alpha & \alpha & \\ \hline & (1 - \frac{1}{2\alpha}) & \frac{1}{2\alpha} \end{array}$$

Second-order methods with two stages

$$\begin{array}{c|ccc} 0 & & & \\ 1/2 & 1/2 & & \\ 1 & -1 & 2 & \\ \hline & 1/6 & 2/3 & 1/6 \end{array}$$

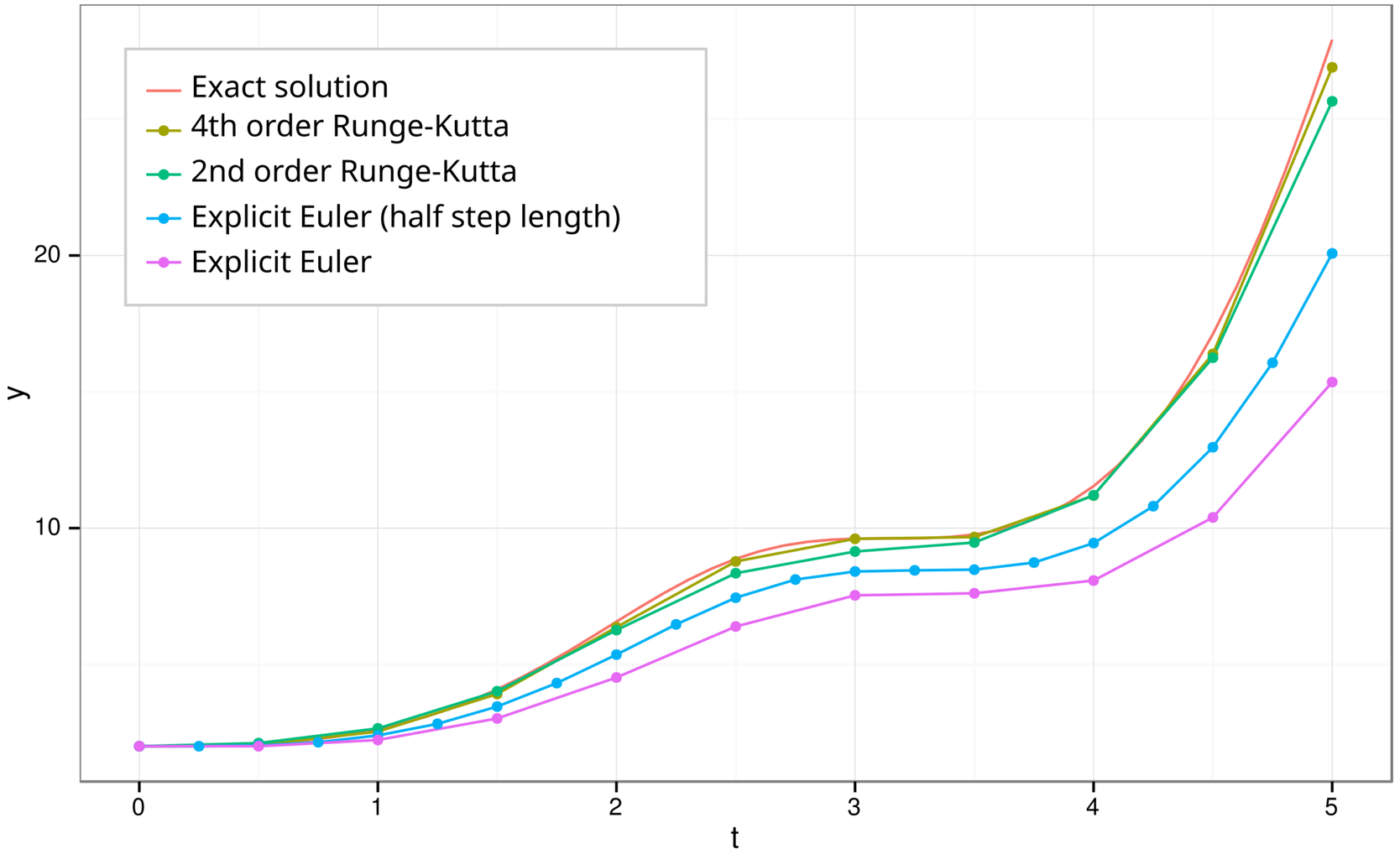
Runge Kutta 3-rd order

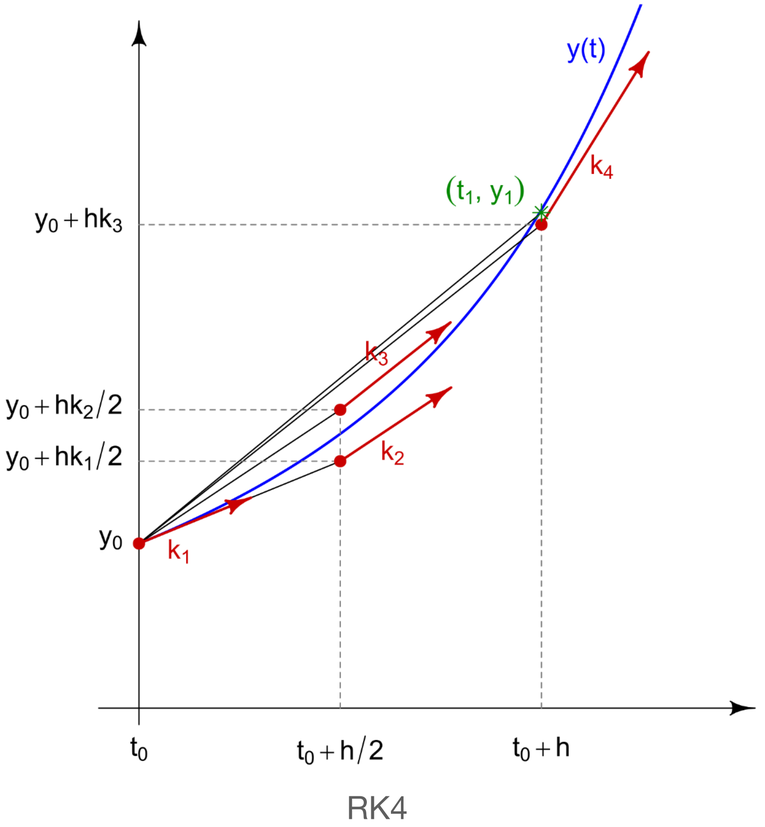
$$\begin{array}{c|ccc} 0 & & & \\ 1/2 & 1/2 & & \\ 1/2 & 0 & 1/2 & \\ 1 & 0 & 0 & 1 \\ \hline & 1/6 & 1/3 & 1/3 \quad 1/6 \end{array}$$

Runge Kutta 4-th order

1. Data-driven solutions of PDEs

b. Discrete time models





RK4

1. Data-driven solutions of PDEs

b. Discrete time models

Implicit Butcher tableau

$$
\begin{array}{c|cccc}
c_1 & a_{11} & a_{12} & \dots & a_{1s} \\
c_2 & a_{21} & a_{22} & \dots & a_{2s} \\
\vdots & \vdots & \vdots & \ddots & \vdots \\
c_s & a_{s1} & a_{s2} & \dots & a_{ss} \\
\hline
 & b_1 & b_2 & \dots & b_s \\
 & b_1^* & b_2^* & \dots & b_s^*
\end{array}
$$

b_i are the *weights* and c_i are the *nodes*

Consistency conditions:

$$\sum_{i=1}^{s} b_i = 1 \text{ and } \sum_{i=1}^{s} b_i^* = 1$$

$$\sum_{j=1}^{s} a_{ij} = c_i$$

Explicit RK unsuitable for the solution of stiff equations because unstable.

Implicit methods are better suited for such scenarios.

1. Data-driven solutions of PDEs

b. Discrete time models

We placed a multi-output neural network prior on $[u^{n+c_1}(x), \ldots, u^{n+c_q}(x), u^{n+1}(x)]$. This results into a **physics-informed neural network** that takes x as an input and outputs $[u_1^n(x), \ldots, u_q^n(x), u_{q+1}^n(x)]$, based on the estimates of $NN_\theta(x)$, where

PINN

$$u_i^n := u^{n+c_i} + \Delta t \sum_{j=1}^{q} a_{ij} \mathcal{N}[u^{n+c_j}], \qquad i = 1, \ldots, q$$

$$u_{q+1}^n := u^{n+1} + \Delta t \sum_{j=1}^{q} b_j \mathcal{N}[u^{n+c_j}] \, .$$

Remark. The key parameters controlling the performance of our discrete time algorithm are the total number of Runge-Kutta stages q and the time-step size Δt.

1. Data-driven solutions of PDEs

b. Discrete time models

The **neural network** takes the spatial coordinate x as input and simultaneously predicts the q intermediate Runge–Kutta stages and the final solution: $NN_\theta(x) = [u_\theta^{n+c_1}(x), \ldots, u_\theta^{n+c_q}(x), u_\theta^{n+1}(x)]$

The **PINN** combines the NN predictions with the RK equations to reconstruct $q+1$ estimates of the initial state $u^n(x)$:

$$u_{i,\theta}^n(x) = u_\theta^{n+c_i}(x) + \Delta t \sum_{j=1}^{q} a_{ij} \mathcal{N}[u_\theta^{n+c_j}](x), \quad i = 1, \ldots, q,$$

$PINN_\theta(x) = [u_{1,\theta}^n(x), \ldots, u_{q,\theta}^n(x), u_{q+1,\theta}^n(x)]$ where

$$u_{q+1,\theta}^n(x) = u_\theta^{n+1}(x) + \Delta t \sum_{j=1}^{q} b_j \mathcal{N}[u_\theta^{n+c_j}](x)\,.$$

Key distinction: The NN predicts the solution at different time stages, while the PINN applies the RK constraints to reconstruct the initial state. During training, all $q+1$ reconstructed values are constrained to match the known $u^n(x)$. Note that the PINN here is the composition of the neural network with the physics-based RK transformations, not a second independently trained nn.

1. Data-driven solutions of PDEs

b. Discrete time models — Training the PINN

Given a set of **observations at the initial time:** $\{(x_k, u^n(x_k))\}_{k=1}^{N_x}$,

we train the neural network by enforcing Runge–Kutta consistency.

Recall that the PINN produces $q+1$ reconstructed initial states: $[u_1^n(x), \ldots, u_q^n(x), u_{q+1}^n(x)]$

According to the RK scheme, all these outputs should approximate the same initial state $u^n(x)$.

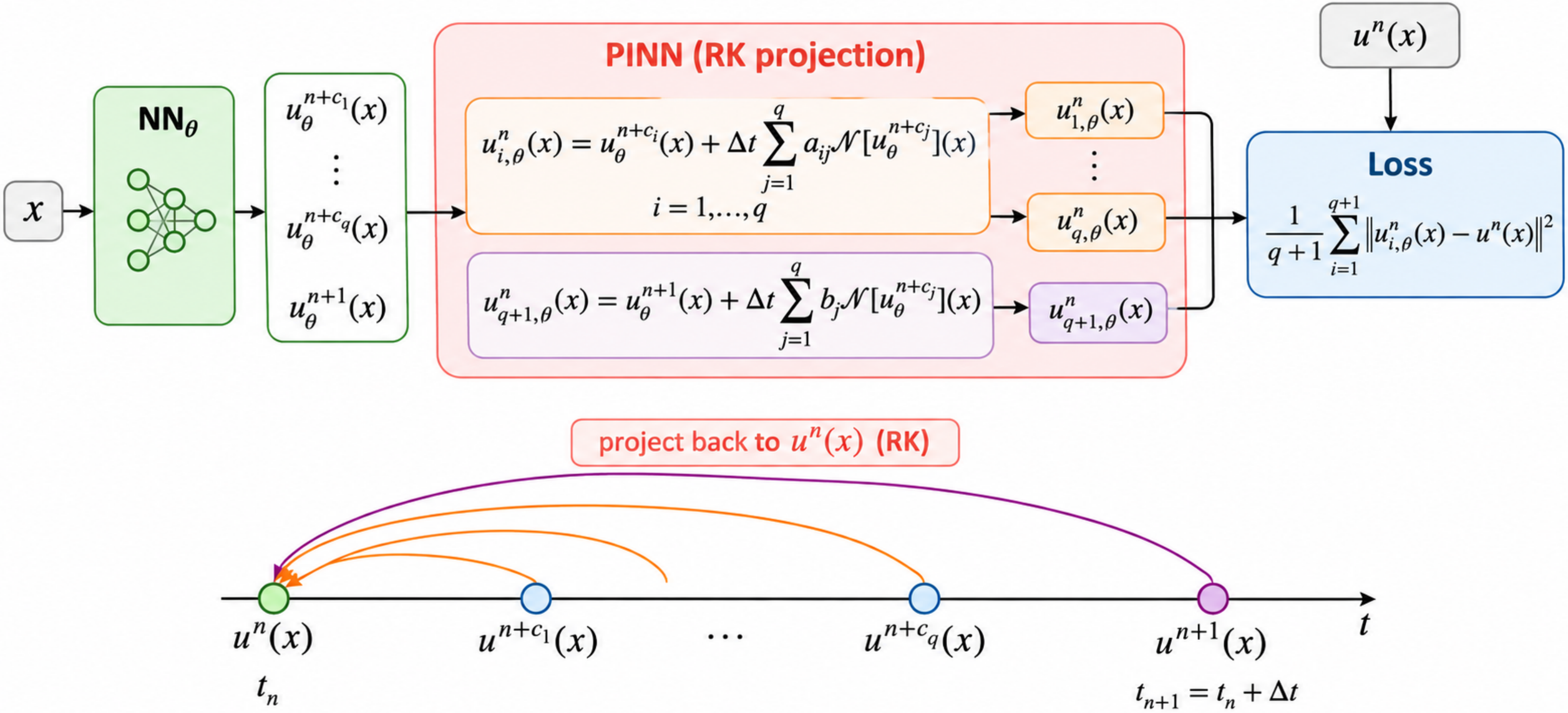
We therefore minimize the following mean squared error loss:

$$\mathscr{L}_{\mathrm{RK}}(\theta) = \frac{1}{N_x(q+1)} \sum_{k=1}^{N_x} \sum_{i=1}^{q+1} \left| u_{i,\theta}^n(x_k) - u^n(x_k) \right|^2$$

When boundary conditions are available, additional constraints can be incorporated:

$$\mathscr{L}(\theta) = \mathscr{L}_{\mathrm{RK}}(\theta) + \lambda_{\mathrm{BC}} \mathscr{L}_{\mathrm{BC}}(\theta)$$

Key idea: Rather than explicitly solving the coupled implicit RK equations, the PINN learns all intermediate stages and the final solution simultaneously by minimizing their consistency error with the known initial state.


x
$\mathbf{NN}_\theta$
$u_\theta^{n+c_1}(x)$
$u_\theta^{n+c_q}(x)$
$u_\theta^{n+1}(x)$
PINN (RK projection)
$u_{i,\theta}^n(x) = u_\theta^{n+c_i}(x) + \Delta t \sum_{j=1}^{q} a_{ij} \mathcal{N}[u_\theta^{n+c_j}](x)$
$i = 1,\ldots,q$
$u_{q+1,\theta}^n(x) = u_\theta^{n+1}(x) + \Delta t \sum_{j=1}^{q} b_j \mathcal{N}[u_\theta^{n+c_j}](x)$
$u_{1,\theta}^n(x)$
$u_{q,\theta}^n(x)$
$u_{q+1,\theta}^n(x)$
$u^n(x)$
Loss
$\frac{1}{q+1} \sum_{i=1}^{q+1} \|u_{i,\theta}^n(x) - u^n(x)\|^2$
project back to $u^n(x)$ (RK)
$u^n(x)$
$u^{n+c_1}(x)$
$\cdots$
$u^{n+c_q}(x)$
$u^{n+1}(x)$
t
t_n
$t_{n+1} = t_n + \Delta t$

Example

Allen Cahn Equation

This example aims to highlight the ability of PINN to handle different types of nonlinearity in the governing partial differential equation.

Consider the Allen–Cahn equation (used for reaction diffusion systems) along with periodic boundary conditions

$$u_t + 0.0001u_{xx} + 5u^3 - 5u = 0 \qquad x \in [-1,1],\ t \in [0,1]$$

$$u(0,\ x) = x^2\cos(\pi x) \longrightarrow \text{initial conditions}$$

$$u(t,\ -1) = u(t,\ 1)$$

$$u_x(t,\ -1) = u_x(t,\ 1) \longrightarrow \text{periodic boundary conditions}$$

The nonlinear operator is given by $\mathcal{N}[u^{n+c_j}] = -0.0001u_{xx}^{n+c_j} + 5(u^{n+c_j})^3 - 5u^{n+c_j}$

Example

Allen Cahn Equation

The shared parameters of the neural network can be learned by minimising the sum of squared errors

$$SSE = SSE_n + SSE_b$$

where

$$SSE_n = \sum_{j=1}^{q+1} \sum_{i=1}^{N_n} | u_j^n(x^{n,i}) - u^{n,i} |^2,$$

$\{x^{n,i}, u^{n,i}\}_{i=1}^{N_n}$ data at time-step t^n

and

$$SSE_b = \sum_{i=1}^{q} | u^{n+c_i}(-1) - u^{n+c_i}(1) |^2 + | u^{n+1}(-1) - u^{n+1}(1) |^2$$

$$+ \sum_{i=1}^{q} | u_x^{n+c_i}(-1) - u_x^{n+c_i}(1) |^2 + | u_x^{n+1}(-1) - u_x^{n+1}(1) |^2 .$$

1. Data-driven solutions of PDEs

b. Discrete time models

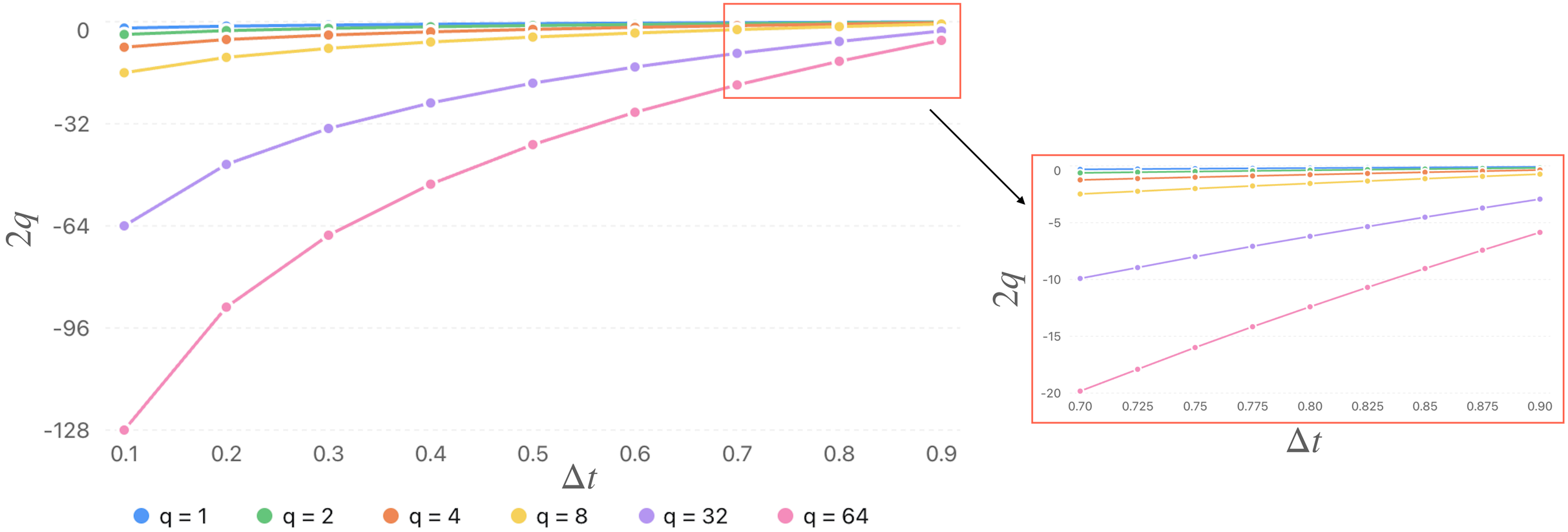
In classical numerical analysis, time-step sizes h are limited by stability constraints for explicit schemes and computational costs for implicit schemes, which increase with the number of Runge–Kutta stages q, often requiring thousands to millions of steps to reach the final time.

In contrast, **discrete-time PINNs use a neural network to simultaneously approximate all stages of an implicit Runge–Kutta scheme**, rather than advancing the solution sequentially in time. This allows the use of high-order schemes with many stages and potentially very large time steps, making it possible to **approximate the entire spatio-temporal solution in a single step**.

b. Discrete time models — Temporal Accuracy

Implicit Gauss–Legendre Runge–Kutta methods with q stages achieve order $2q$, yielding a **theoretical global temporal error** of: $E_{\text{time}} = O(\Delta t^{2q})$

Increasing the number of RK stages can dramatically reduce the temporal discretization error, potentially allowing large time steps while maintaining high accuracy.



Example

Allen Cahn Equation

$\{x^{n,i}, u^{n,i}\}_{i=1}^{N_n}$ data at time-step t^n

Generated a training and test data-set set by simulating the Allen-Cahn equation starting from an initial condition $u(0,x) = x^2 \cos(\pi x)$ and assuming periodic boundary conditions $u(t,-1) = u(t,1)$ and $u_x(t,-1) = u_x(t,1)$.

True solution. We have integrated the Allen-Cahn equation up to a final time $t = 1.0$ using a fourth-order explicit Runge-Kutta temporal integrator with time-step $\Delta t = 10^{-5}$.

Training data. Training data-set consists of $N_n = 200$ initial data points that are randomly sub-sampled from the exact solution at time $t = 0.1$.

Predictive task. Our goal is to predict the solution at time $t = 0.9$ using a single time-step with size $\Delta t = 0.8$.

Example

Allen Cahn Equation

$\{x^{n,i}, u^{n,i}\}_{i=1}^{N_n}$ data at time-step t^n

Learning scheme.

Discrete time physics-informed neural network with 4 hidden layers and 200 neurons per layer.

The output layer predicts 101 quantities of interest corresponding to the $q = 100$ Runge-Kutta stages, $u^{n+c_i}(x)$, $i = 1, \ldots, q$, and the solution at final time $u^{n+1}(x)$.

The theoretical error estimates for this scheme predict a temporal error accumulation of $O(\Delta t^{2q})$, which in our case translates into an error way below machine precision, i.e., $\Delta t^{2q} = 0.8^{200} \approx 10^{-20}$.

Remarkably, starting from smooth initial data at $t = 0.1$ we can predict the nearly discontinuous solution at $t = 0.9$ in a single time-step with a relative $\mathbb{L}_2$ error of $6.99 \cdot 10^{-3}$. This error is entirely attributed to the neural network's capacity to approximate $u(t, x)$, as well as to the degree that the sum of squared errors loss allows interpolation of the training data.

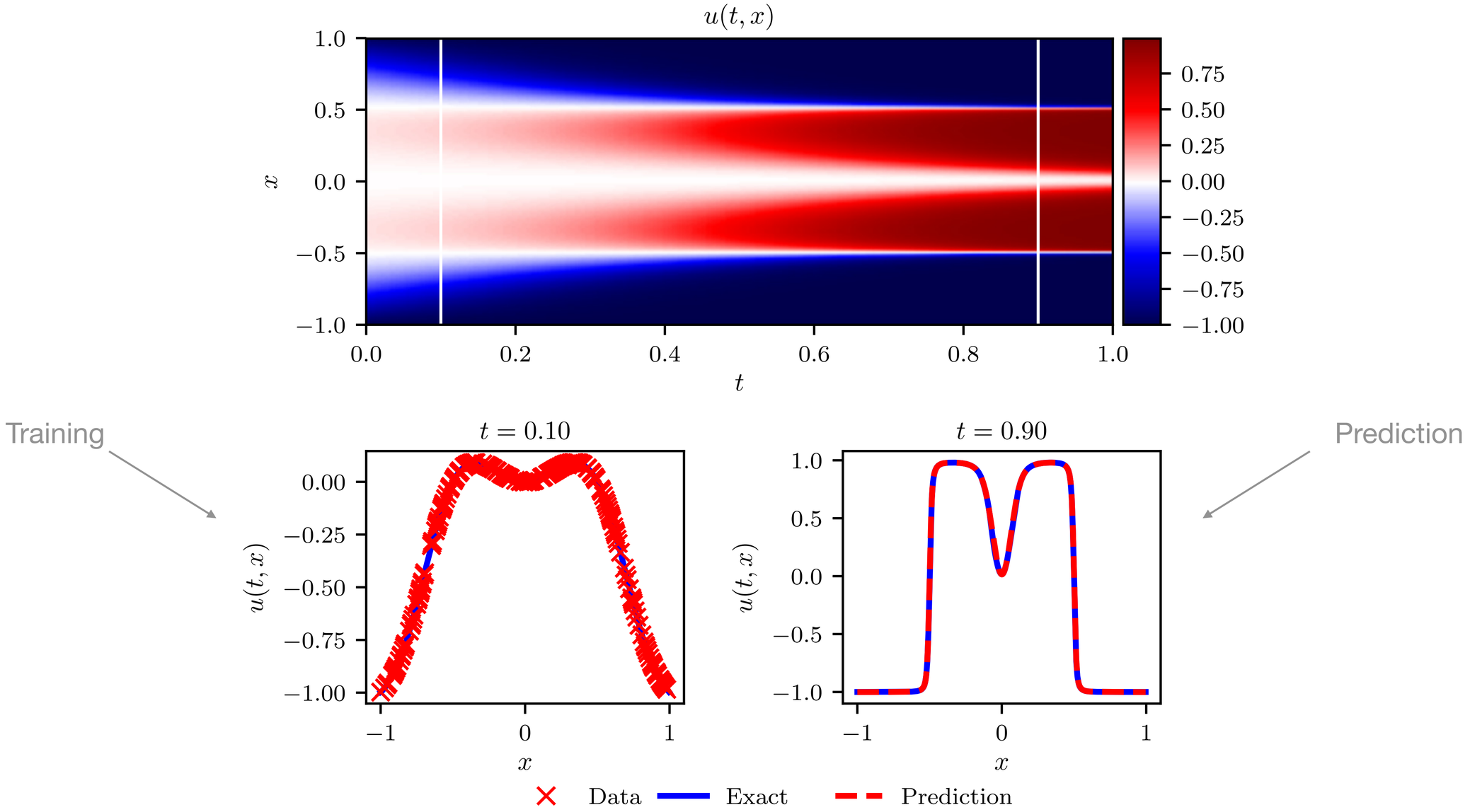


Fig. 2. *Allen–Cahn equation: Top:* Solution $u(t,x)$ along with the location of the initial training snapshot at $t = 0.1$ and the final prediction snapshot at $t = 0.9$. *Bottom:* Initial training data and final prediction at the snapshots depicted by the white vertical lines in the top panel. The relative $\mathbb{L}_2$ error for this case is $6.99 \cdot 10^{-3}$. (For interpretation of the colors in the figure(s), the reader is referred to the web version of this article.)

1.b. Demo session

https://github.com/maziarraissi/PINNs

https://github.com/maziarraissi/PINNs/tree/master/main/discrete_time_inference

2. Data-driven identification of PDE

a. Continuous time models

Let $f := u_t + \mathcal{N}[u; \lambda]$.

Proceed by approximate $u(t, x)$ by a deep neural network — resulting into a PINN $f(t, x)$. This network can be derived by applying the chain rule for differentiating compositions of functions using automatic differentiation.

It is worth highlighting that the parameters λ of the differential operator $\mathcal{N}[u; \lambda]$ turn into parameters of the PINN $f(t, x)$.

Example

Navier-Stokes Equation

Flow of an incompressible fluid described by the Navier–Stokes equations. Navier–Stokes equations describe the physics of many phenomena: weather, ocean currents, water flow in a pipe, blood flow and air flow around a wing.

Navier-Stokes in 2D

$$u_t + \lambda_1(uu_x + vu_y) = -p_x + \lambda_2(u_{xx} + u_{yy})$$
$$v_t + \lambda_1(uv_x + vv_y) = -p_y + \lambda_2(v_{xx} + v_{yy})$$

streamwise vs transverse velocities

where $u(t, x, y)$ and $v(t, x, y)$ denote respectively the x and y-component of the velocity field and $p(t, x, y)$ the pressure. Here, $\lambda = (\lambda_1, \lambda_2)$ are the unknown parameters

Example

Navier-Stokes Equation

continuity equation for incompressible fluids that describes the conservation of mass of the fluid

Solutions to the Navier–Stokes equations are searched in the set of divergence-free functions: $u_x + v_y = 0$

We make the assumption that $u = \psi_y$ and $v = -\psi_x$ for some latent function $\psi(t, x, y)$

under this assumption, the continuity equation is automatically satisfied

Given noisy measurements $\{t^i, x^i, y^i, u^i, v^i\}_{i=1}^N$ of the velocity field, we are interested in learning the parameters λ as well as the pressure $p(t, x, y)$.

We define $f(t, x, y)$ and $g(t, x, y)$ to be given by

$$f := u_t + \lambda_1(uu_x + vu_y) + p_x - \lambda_2(u_{xx} + u_{yy})$$

$$g := v_t + \lambda_1(uv_x + vv_y) + p_y - \lambda_2(v_{xx} + v_{yy})$$

$\longrightarrow [f(t, x, y), g(t, x, y)]$ is a PINN

and proceed by jointly approximating $[\psi(t, x, y), p(t, x, y)]$ using a single neural network with two outputs.

Example

Navier-Stokes Equation

The parameters λ together with the parameters of the neural networks $[\psi(t,x,y), p(t,x,y)]$ can be trained by minimizing the mean squared error loss

$$MSE := \frac{1}{N}\sum_{i=1}^{N}\left(|\,u(t^i,x^i,y^i) - u^i\,|^2 + |\,v(t^i,x^i,y^i) - v^i\,|^2\right) + \frac{1}{N}\sum_{i=1}^{N}\left(|\,f(t^i,x^i,y^i)\,|^2 + |\,g(t^i,x^i,y^i)\,|^2\right)$$

$u = \psi_y$ $\qquad$ $v = -\psi_x$

Given scattered and potentially noisy data on the stream-wise $u(t,x,y)$ and transverse $v(t,x,y)$ velocity components, our goal is to identify the unknown parameters λ_1 and λ_2 as well as to obtain a qualitatively accurate reconstruction of the entire pressure field $p(t,x,y)$.

Example

Navier-Stokes - training data

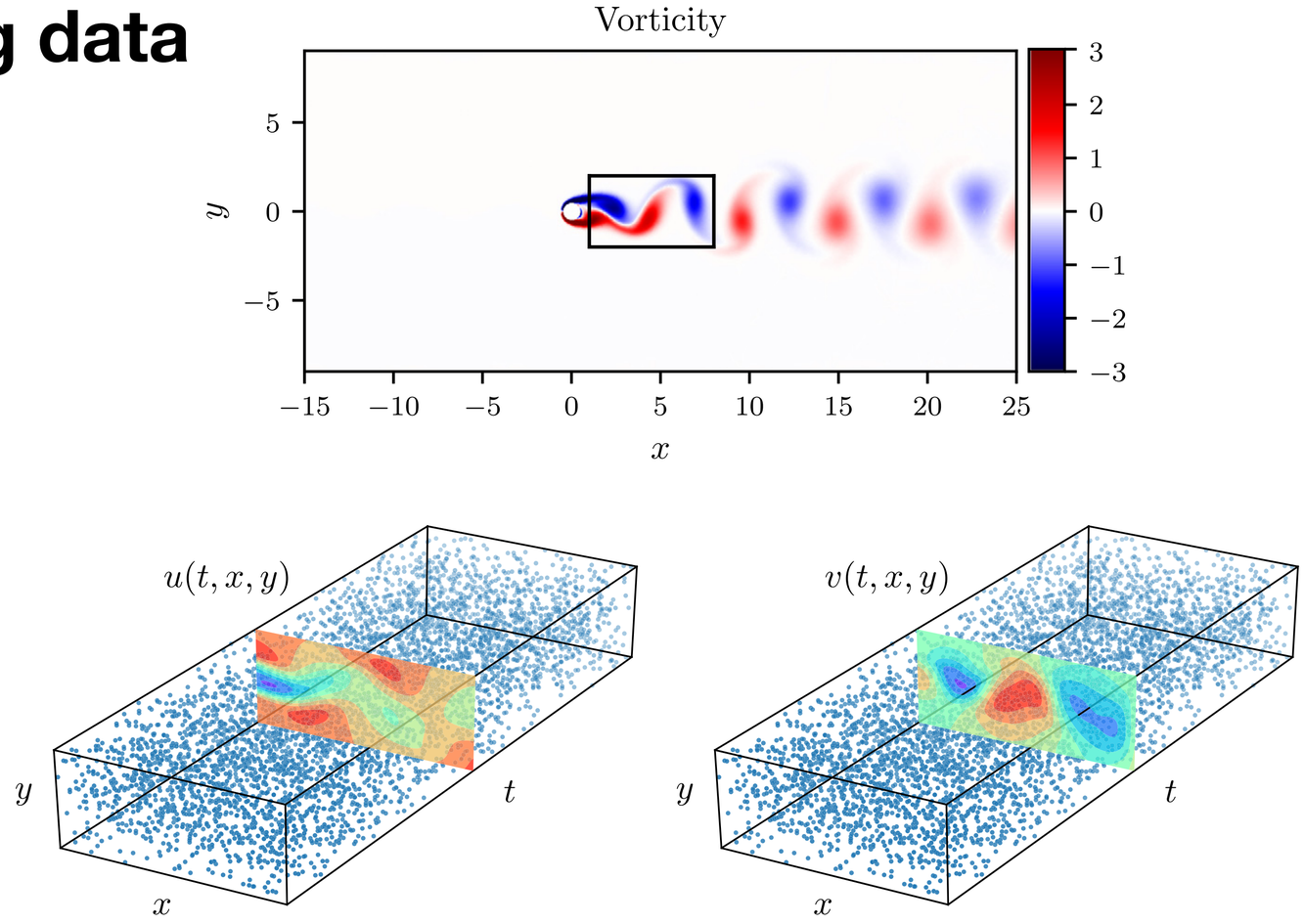
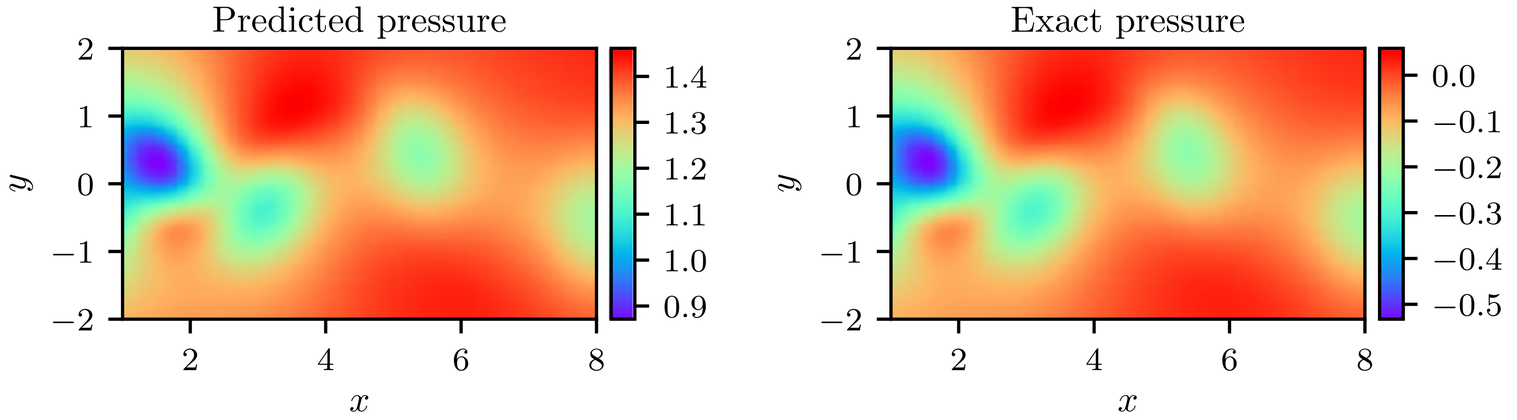


Fig. 3. *Navier–Stokes equation: Top:* Incompressible flow and dynamic vortex shedding past a circular cylinder at $Re = 100$. The spatio-temporal training data correspond to the depicted rectangular region in the cylinder wake. *Bottom:* Locations of training data-points for the stream-wise and transverse velocity components, $u(t, x, y)$ and $v(t, x, t)$, respectively.

https://github.com/maziarraissi/PINNs/tree/master/main/continuous_time_identification

Example
Navier-Stokes - Results

- The PINN identify the unknown parameters λ_1 and λ_2 with very high accuracy even when the training data was corrupted with noise.
- The PINN is also able to accurately predict the entire pressure field $p(t, x, y)$ in the absence of any training data on the pressure itself.



Correct PDE	$u_t + (uu_x + vu_y) = -p_x + 0.01(u_{xx} + u_{yy})$ $v_t + (uv_x + vv_y) = -p_y + 0.01(v_{xx} + v_{yy})$
Identified PDE (clean data)	$u_t + 0.999(uu_x + vu_y) = -p_x + 0.01047(u_{xx} + u_{yy})$ $v_t + 0.999(uv_x + vv_y) = -p_y + 0.01047(v_{xx} + v_{yy})$
Identified PDE (1% noise)	$u_t + 0.998(uu_x + vu_y) = -p_x + 0.01057(u_{xx} + u_{yy})$ $v_t + 0.998(uv_x + vv_y) = -p_y + 0.01057(v_{xx} + v_{yy})$

Fig. 4. *Navier–Stokes equation: Top:* Predicted versus exact instantaneous pressure field $p(t, x, y)$ at a representative time instant. By definition, the pressure can be recovered up to a constant, hence justifying the different magnitude between the two plots. This remarkable qualitative agreement highlights the ability of *physics-informed neural networks* to identify the entire pressure field, despite the fact that no data on the pressure are used during model training. *Bottom:* Correct partial differential equation along with the identified one obtained by learning λ_1, λ_2 and $p(t, x, y)$.

2. Data-driven identification of PDE

a. Continuous time models

Limitation

The method so far assumes **availability of scattered data throughout the entire spatio-temporal domain**. However, in many cases of practical interest, one may only be able to observe the system at distinct time instants.

We now introduce a different approach that tackles the data-driven discovery problem using only two data snapshots. We will see how, by leveraging the classical Runge-Kutta time-stepping schemes, one can construct discrete time physics-informed neural networks that can retain high predictive accuracy even when the temporal gap between the data snapshots is very large.

2. Data-driven identification of PDE

b. Discrete time models

Apply the general form of Range-Kutta methods with q stages to the PDE and obtain

$$u^{n+c_i} = u^n - \Delta t \sum_{j=1}^{q} a_{ij} \mathcal{N}[u^{n+c_j}; \lambda], \; i = 1, \ldots, q$$

$$u^{n+1} = u^n - \Delta t \sum_{j=1}^{q} b_j \mathcal{N}[u^{n+c_j}; \lambda]$$

Here, $u^{n+c_j}(x) = u(t^n + c_j \Delta t, x)$ is the hidden state of the system at time $t^n + c_j \Delta t$ for $j = 1, \ldots, q$. This general form encapsulates both implicit and explicit time-stepping schemes, depending on the choice of the parameters $\{a_{ij}, b_j, c_j\}$.

We proceed by placing a multi-output neural network prior on $[u^{n+c_1}(x), \ldots, u^{n+c_q}(x), u^{n+1}(x)]$. This results into **two** physics-informed neural networks $[u_1^n(x), \ldots, u_q^n(x), u_{q+1}^n(x)]$ and $[u_1^{n+1}(x), \ldots, u_q^{n+1}(x), u_{q+1}^{n+1}(x)]$.

2. Data-driven identification of PDE

b. Discrete time models

We proceed by placing a multi-output neural network prior on $[u^{n+c_1}(x), \ldots, u^{n+c_q}(x), u^{n+1}(x)]$.

This results into two physics-informed neural networks $[u_1^n(x), \ldots, u_q^n(x), u_{q+1}^n(x)]$ and

$[u_1^{n+1}(x), \ldots, u_q^{n+1}(x), u_{q+1}^{n+1}(x)]$,

where

$$u_i^n := u^{n+c_i} + \Delta t \sum_{j=1}^{q} a_{ij} \mathcal{N}[u^{n+c_j}; \lambda], \qquad i = 1, \ldots, q$$

$$u_i^{n+1} := u^{n+c_i} + \Delta t \sum_{j=1}^{q} (a_{ij} - b_j) \mathcal{N}[u^{n+c_j}; \lambda], \qquad i = 1, \ldots, q$$

Data-driven identification of PDE

b. Discrete time models

Given noisy measurements at two distinct temporal snapshots $\{\mathbf{x}^n, \mathbf{u}^n\}$ and $\{\mathbf{x}^{n+1}, \mathbf{u}^{n+1}\}$ of the system at times t^n and t^{n+1}, respectively, the shared parameters of the neural networks along with the parameters λ of the differential operator can be trained by minimizing the sum of squared errors

$$SSE = SSE_n + SSE_{n+1}$$

where

$$SSE_n = \sum_{j=1}^{q} \sum_{i=1}^{N_n} | u_j^n(x^{n,i}) - u^{n,i} |^2$$

and

$$SSE_{n+1} = \sum_{j=1}^{q} \sum_{i=1}^{N_{n+1}} | u_j^{n+1}(x^{n+1,i}) - u^{n+1,i} |^2$$

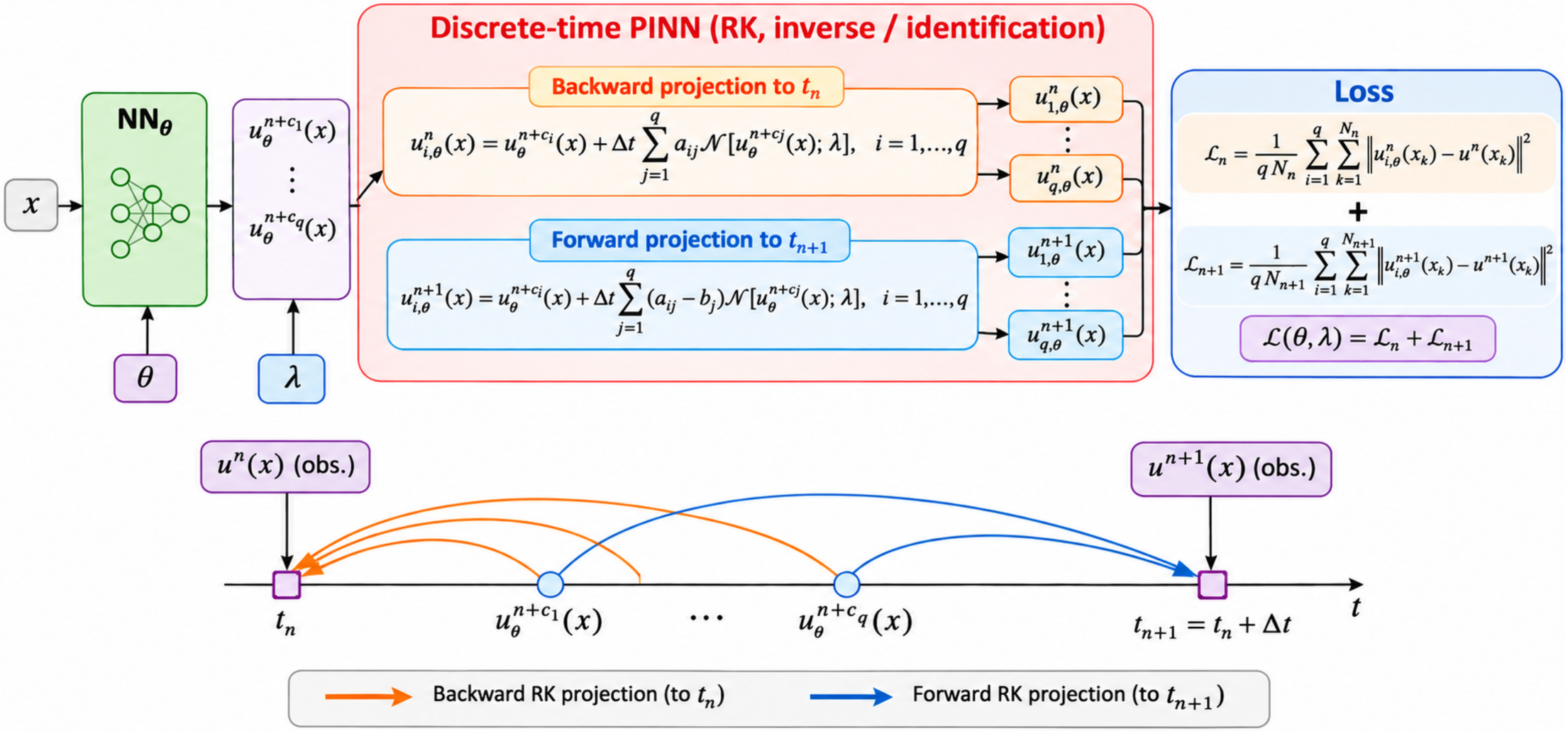
Here,

$$\mathbf{x}^n = \{x^{n,i}\}_{i=1}^{N_n} \qquad \mathbf{u}^n = \{u^{n,i}\}_{i=1}^{N_n}$$

$$\mathbf{x}^{n+1} = \{x^{n+1,i}\}_{i=1}^{N_{n+1}} \qquad \mathbf{u}^{n+1} = \{u^{n+1,i}\}_{i=1}^{N_{n+1}}$$

2. Data-driven identification of PDE

b. Discrete time models



Example

Korteweg de Vries Equation

Mathematical model of waves on shallow water surfaces (Burgers equation with an added dispersive term)

$$u_t + \lambda_1 u u_x + \lambda_2 u_{xxx} = 0$$

with (λ_1, λ_2) being the unknown parameters.

The nonlinear operator $\mathcal{N}[u^{n+c_j}; \lambda] = \lambda_1 u_x^{n+c_j} - \lambda_2 u_{xxx}^{n+c_j}$.

The shared parameters of the neural networks along with the parameters (λ_1, λ_2) of the KdV equation can be learned by minimizing the sum of squared errors SSE.

Example

Korteweg de Vries Equation

To obtain a set of training and test data we start from an initial condition $u(0,x) = cos(\pi x)$ and assuming periodic boundary conditions, we have integrated equation up to a final time $t = 1.0$ with a fourth-order explicit Runge–Kutta temporal integrator with time-step $\Delta t = 10^{-6}$.

Training data. Using this data-set, we then extract two solution snapshots at time $t^n = 0.2$ and $t^{n+1} = 0.8$, and randomly sub-sample them using $N_n = 199$ and $N_{n+1} = 201$ to generate a training data-set.

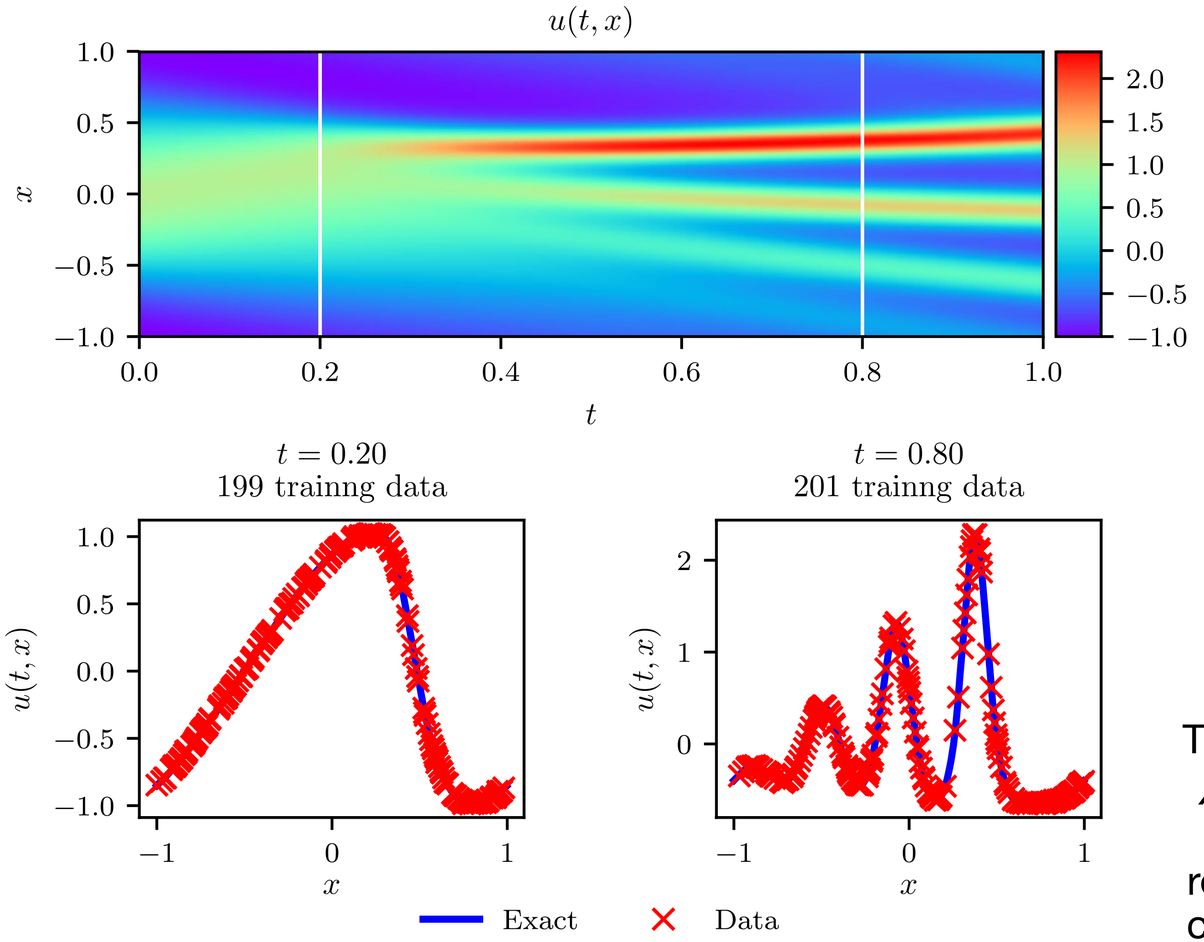
Learning scheme. We then use these data to train a discrete time physics-informed neural network by minimizing the sum of squared error loss SSE using L-BFGS.

The network architecture used here comprises of 4 hidden layers, 50 neurons per layer, and an output layer predicting the solution at the q Runge–Kutta stages, i.e., u^{n+c_j}, for $j = 1,\ldots,q$ where q is empirically chosen to yield a temporal error accumulation of the order of machine precision ϵ by setting $q = 0.5\log(\epsilon)/\log(\Delta t)$ where the time-step for this example is $\Delta t = 0.6$.

Example

KdV - Results

https://github.com/maziarraissi/PINNs/tree/master/main/discrete_time_identification



The complex nonlinear dynamics causes dramatic differences in the form of the solution between the two reported snapshots.

Despite these differences, and the large temporal gap between the two training snapshots, our method is able to correctly identify the unknown parameters regardless of whether the training data is corrupted with noise or not.

Correct PDE	$u_t + uu_x + 0.0025u_{xxx} = 0$
Identified PDE (clean data)	$u_t + 1.000uu_x + 0.0025002u_{xxx} = 0$
Identified PDE (1% noise)	$u_t + 0.999uu_x + 0.0024996u_{xxx} = 0$

The error in estimating λ_1 and λ_2 is 0.023%, and 0.006%, respectively, while the case with 1% noise in the training data returns errors of 0.057%, and 0.017%, respectively.

Fig. 5. *KdV equation: Top:* Solution $u(t, x)$ along with the temporal locations of the two training snapshots. *Middle:* Training data and exact solution corresponding to the two temporal snapshots depicted by the dashed vertical lines in the top panel. *Bottom:* Correct partial differential equation along with the identified one obtained by learning λ_1, λ_2.

PINN — Summary

A class of **universal** function **approximators** capable of **encoding** any underlying **physical laws** that govern a given data-set and can be described by partial differential equations.

We design data-driven algorithms for inferring solutions (and estimate parameters) to general nonlinear partial differential equations and constructing computationally efficient **physics-informed surrogate models**.

	Forward Problem (Solution)	**Inverse Problem (Identification)**
Continuous-time PINNs	• Known PDE parameters • Initial condition $u(0,x)$ • Boundary conditions • Space-time collocation points t, x	• Solution observations $u(t_i, x_i)$, typically at multiple times • Space-time collocation points t, x • Initial/boundary conditions (if available)
Discrete-time PINNs	• Known PDE parameters • Initial solution $u(t_n, x)$ • Boundary conditions (if required)	• Solution observations at two or more times, e.g. $u(t_n, x)$ and $u(t_{n+1}, x)$ • Boundary conditions (if required)

Wang, S., Sankaran, S., & Perdikaris. “Respecting Causality for Training Physics-Informed Neural Networks”. Computer Methods in Applied Mechanics and Engineering, 2024.

Causal PINNs: Motivation

Consider a time-dependent PDE:

$$u_t + \mathcal{N}[u] = 0, \qquad x \in \Omega,\ t \in [0,T]$$
$$u(0,x) = g(x), \qquad x \in \Omega$$

with suitable boundary conditions.

Physical systems evolve causally: the solution at later times depends on the evolution of the system at earlier times.

Traditional numerical solvers naturally respect this temporal ordering:

$$u(t_0) \longrightarrow u(t_1) \longrightarrow \cdots \longrightarrow u(t_N)\,.$$

However, **standard continuous-time PINNs approximate the solution over the entire space-time domain simultaneously**, without explicitly enforcing this temporal ordering during training.

This can cause serious optimization difficulties, particularly for long-time simulations and chaotic dynamics.

Causal PINNs: The Problem with Standard PINNs

Recall that a continuous-time PINN approximates: $u_\theta(t,x) \approx u(t,x)$ by minimizing:

$$\mathscr{L}(\theta) = \lambda_{\mathrm{IC}}\mathscr{L}_{\mathrm{IC}} + \lambda_{\mathrm{BC}}\mathscr{L}_{\mathrm{BC}} + \mathscr{L}_{\mathrm{PDE}}$$

where the PDE residual is: $\mathscr{R}_\theta(t,x) = \dfrac{\partial u_\theta}{\partial t} + \mathscr{N}[u_\theta]\,.$

The standard residual loss is: $\mathscr{L}_{\mathrm{PDE}} = \dfrac{1}{N_r}\displaystyle\sum_{i=1}^{N_r} |\,\mathscr{R}_\theta(t_i,x_i)\,|^2\,.$

The problem: Collocation points from early and late times contribute to the optimization simultaneously.

Consequently, **the network may reduce the PDE residual at later times before accurately learning the initial conditions and early-time dynamics.**

Note: A small PDE residual at later times does not necessarily imply that the solution is correct!!

Causal PINNs: Example

Constant solutions

Consider a simple ODE: $\frac{du}{dt} = -\lambda u, \qquad u(0) = 1$

whose exact solution is: $u(t) = e^{-\lambda t}$.

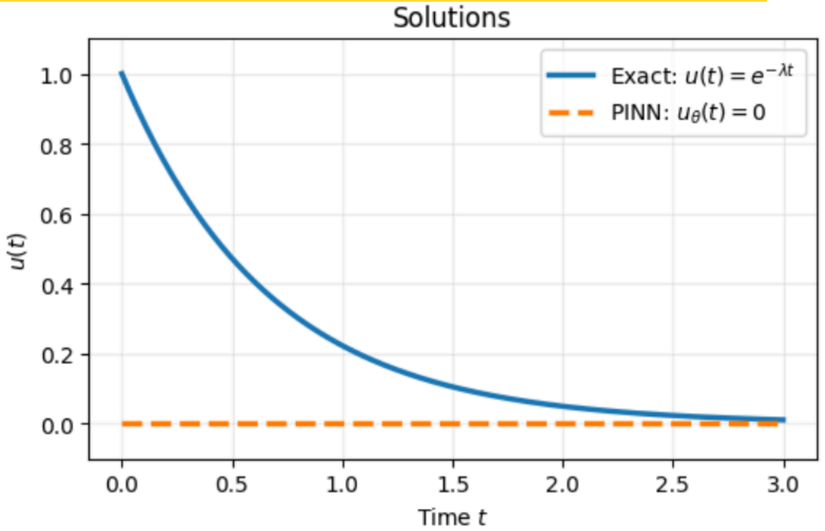
Suppose the neural network predicts: $u_\theta(t) = 0$.

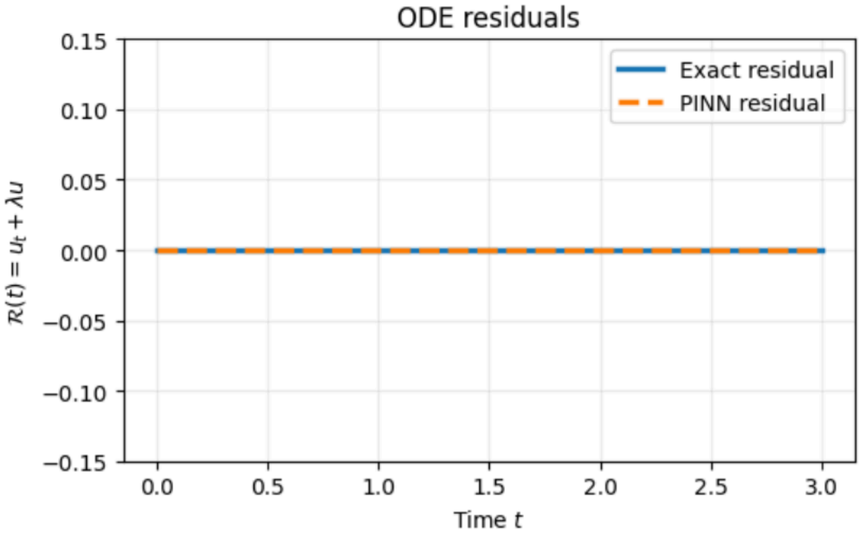
Then: $\mathscr{R}_\theta(t) = \frac{du_\theta}{dt} + \lambda u_\theta = 0.$

Therefore: $\mathscr{L}_{\text{PDE}} = 0$ even though the prediction violates the initial condition: $u_\theta(0) = 0 \neq 1$

The zero function satisfies the differential equation but not the specified initial-value problem.

Key insight: Minimizing the PDE residual alone does not identify the correct solution. Initial conditions must be enforced effectively, and **errors at early times can affect the entire future trajectory.**





Causal PINNs: Example

Learning Oscillatory Dynamics

Problem: Learning Long-Time Dynamics

Consider a harmonic oscillator governed by:

$$\frac{du}{dt} = v, \qquad \frac{dv}{dt} = -\omega^2 u, \qquad u(0) = 1, \quad v(0) = 0.$$

The exact solution is: $u(t) = \cos(\omega t), \qquad v(t) = -\omega \sin(\omega t)\,.$

Goal: Train a PINN to approximate the solution over a long time interval containing several oscillations.

A standard PINN represents both state variables: $NN_\theta(t) = [u_\theta(t), v_\theta(t)]$

Challenge: The network must learn a complex temporal evolution while satisfying the initial conditions and governing equations.

Causal PINNs: Example

Learning Oscillatory Dynamics

The PINN enforces the governing equations through two residuals:

$$\mathscr{R}_1(t) = \frac{du_\theta}{dt} - v_\theta,$$

$$\mathscr{R}_2(t) = \frac{dv_\theta}{dt} + \omega^2 u_\theta\,.$$

The training objective is: $\mathscr{L} = \lambda_{\text{IC}}\mathscr{L}_{\text{IC}} + \mathscr{L}_{\text{PDE}},$

where $\mathscr{L}_{\text{PDE}} = \frac{1}{N}\sum_{i=1}^{N}\left[\,|\,\mathscr{R}_1(t_i)\,|^2 + |\,\mathscr{R}_2(t_i)\,|^2\right].$

The problem: The network minimizes the residual across the entire temporal domain simultaneously.

Even when early-time dynamics are poorly approximated, the training algorithm may focus on large residuals at much later times.

This does not reflect the causal structure of the dynamical system.

Causal PINNs: Example

Learning Oscillatory Dynamics

Key insight: Large residuals at later times can dominate the training objective, even though the accuracy of the evolution depends on correctly learning earlier dynamics.

Divide the temporal domain into three regions: $[0,T] = \underbrace{[0,T/3]}_{\text{Early}} \cup \underbrace{[T/3,2T/3]}_{\text{Middle}} \cup \underbrace{[2T/3,T]}_{\text{Late}}$.

Suppose that, during training, the PDE residual losses are:

Time region	**Residual loss**
Early	0.10
Middle	1.00
Late	5.00

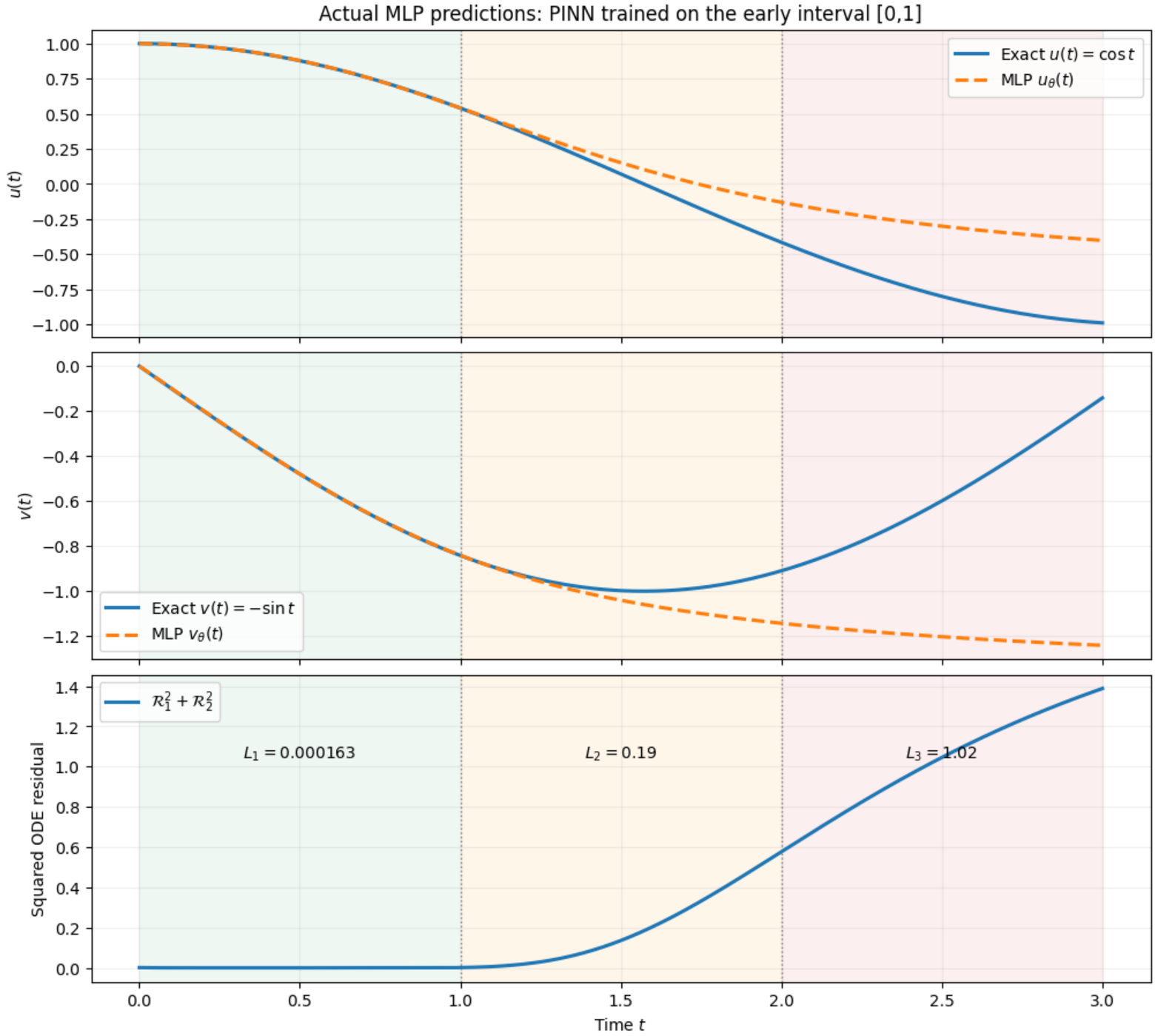
The standard PINN minimizes: $\mathscr{L}_{\text{PDE}} = \dfrac{0.10 + 1.00 + 5.00}{3} = 2.033.$

The late-time region contributes approximately 82% of the total PDE residual loss.

Consequently, the optimizer may devote substantial effort to reducing late-time errors before earlier dynamics have been accurately learned.

Causal PINNs:
Example

Learning
Oscillatory
Dynamics



Causal PINNs: Introducing Temporal Causality into the Loss

The idea of Causal PINNs is to prioritize learning earlier times before later times.

Divide the temporal domain into N_t ordered time slices: $0 = t_0 < t_1 < \cdots < t_{N_t} = T$.

For each time slice t_i, define the corresponding PDE residual loss:

$$\mathscr{L}_i(\theta) = \frac{1}{N_x} \sum_{j=1}^{N_x} |\, \mathscr{R}_\theta(t_i, x_j) \,|^2$$

The standard PINN minimizes: $\mathscr{L}_{\text{PDE}} = \frac{1}{N_t} \sum_{i=1}^{N_t} \mathscr{L}_i$.

Every temporal slice receives the same explicit weight, regardless of whether the preceding dynamics have been learned.

Proposed solution: Introduce temporal weights that depend on the errors at earlier times.

Causal PINNs: Causal Weighting

Replace the standard PDE residual loss with: $$\mathscr{L}_{\text{causal}}(\theta) = \frac{1}{N_t}\sum_{i=1}^{N_t} w_i \mathscr{L}_i(\theta)$$

The weights should satisfy the following principle:

> **A time slice should receive a large weight only when the earlier time slices have been learned accurately.**

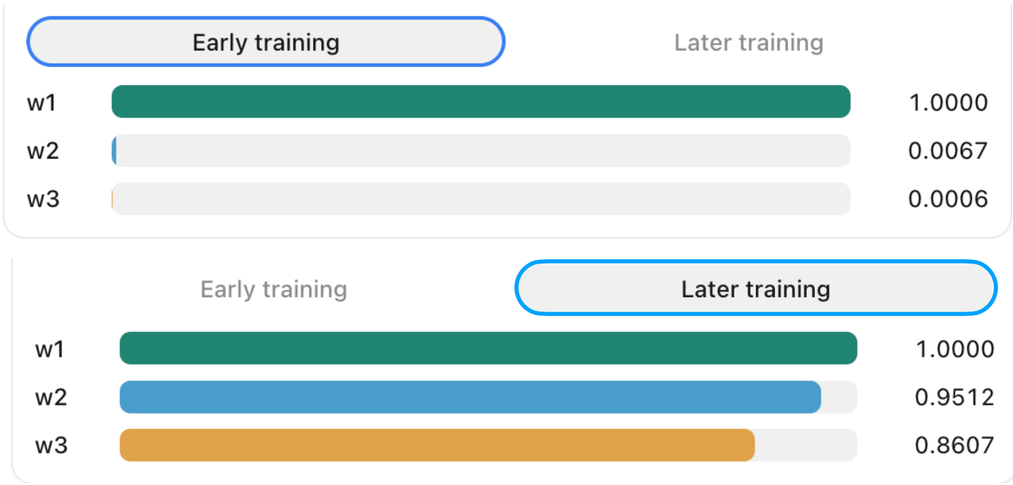
For instance: $$w_i = \exp\left(-\epsilon \sum_{k=1}^{i-1} \mathscr{L}_k\right), \qquad w_1 = 1$$

where $\epsilon > 0$ controls the strength of causal weighting.

- **Large previous residuals**: $w_i \approx 0$, suppressing later-time losses.
- **Small previous residuals**: $w_i \approx 1$, allowing later-time losses to contribute.
- **First time slice**: $w_1 = 1$, so it is always prioritized.

The weights are recomputed during training from the current residuals.

Causal PINNs: Causal Weighting



Consider three time slices and choose $\epsilon = 5$.

Their weights are:

$$w_1 = 1$$

$$w_2 = e^{-5\mathscr{L}_1}$$

$$w_3 = e^{-5(\mathscr{L}_1+\mathscr{L}_2)}.$$

Suppose the initial residuals are: $\mathscr{L}_1 = 1, \qquad \mathscr{L}_2 = 0.5.$

We obtain: $w_1 = 1, \qquad w_2 \approx 0.0067, \qquad w_3 \approx 0.00055.$

The network initially focuses almost entirely on the first time slice.

As training progresses, suppose: $\mathscr{L}_1 = 0.01, \qquad \mathscr{L}_2 = 0.02.$

Then: $w_1 = 1, \qquad w_2 \approx 0.951, \qquad w_3 \approx 0.861.$

As early-time errors decrease, the optimization progressively incorporates later-time dynamics.

Causal PINNs: Loss Function

The neural network architecture remains unchanged: $NN_\theta(t,x) = u_\theta(t,x)$.

Only the training objective is modified: $\mathscr{L}(\theta) = \lambda_{\text{IC}}\mathscr{L}_{\text{IC}} + \lambda_{\text{BC}}\mathscr{L}_{\text{BC}} + \mathscr{L}_{\text{causal}}$

with $\mathscr{L}_{\text{causal}} = \frac{1}{N_t}\sum_{i=1}^{N_t} \underbrace{e^{-\epsilon \sum_{k<i} \mathscr{L}_k}}_{\text{Causal weight}} \underbrace{\mathscr{L}_i}_{\text{PDE residual at } t_i}$

Important implementation detail: The causal weights are treated as constants during each gradient update (stop-gradient).

Hence, the residual-loss gradient used for the update is: $\nabla_\theta \mathscr{L}_{\text{causal}} = \frac{1}{N_t}\sum_{i=1}^{N_t} w_i \nabla_\theta \mathscr{L}_i$

where **gradients are not propagated through** w_i.

This ensures that the weights control the relative importance of each time slice without introducing additional gradients through their exponential dependence on earlier losses.

Causal PINNs: Training Algorithm

Note: High causal weights are a useful training diagnostic, but they do not guarantee that the learned solution is globally accurate.

Input: Initial and boundary conditions, collocation points grouped into ordered time slices, and a sequence of causal parameters $\epsilon_1, \ldots, \epsilon_K$.

Algorithm:

1. Initialize network parameters θ.
2. Choose a causal parameter ϵ from an increasing schedule.
3. Evaluate the PDE residual losses $\mathscr{L}_1, \ldots, \mathscr{L}_{N_t}$.
4. Compute the causal weights: $w_i = \exp\left(-\epsilon \sum_{k<i} \mathscr{L}_k\right)$.
5. Compute the weighted loss, detach the weights, and update θ using gradient descent.
6. Repeat until the temporal weights approach one, then continue with the next ϵ.

Why not start with a large ϵ?

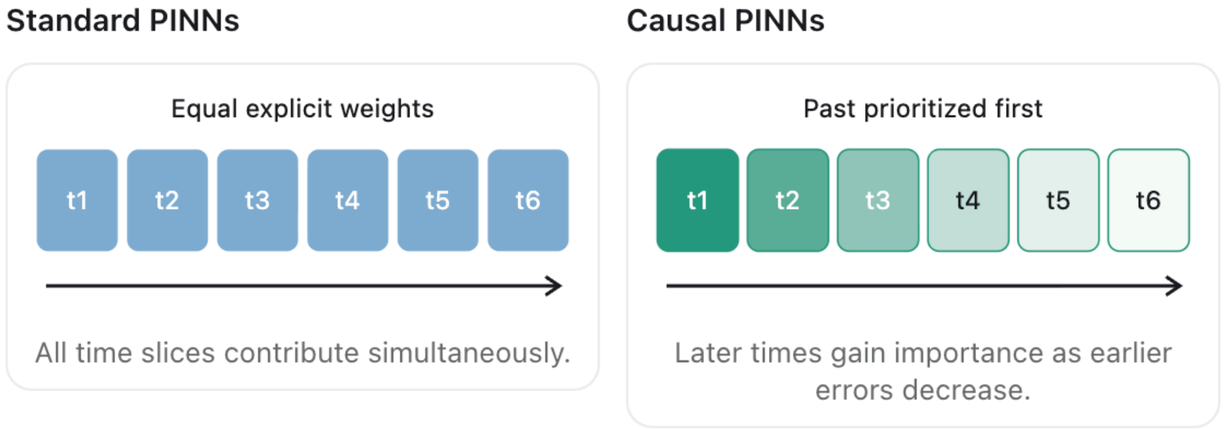
Because we risk prematurely suppressing the learning of later-time dynamics.

Convergence criterion: $\min_i w_i > \delta, \qquad \delta \approx 1$

This indicates that the accumulated earlier residuals are sufficiently small relative to ϵ.

The original method uses an increasing schedule of ϵ values to progressively enforce stricter causal ordering.

Causal PINNs



Main advantages

- Better alignment between optimization and the temporal evolution of the physical system.
- Improved training on challenging time-dependent PDEs.
- No need to change the network architecture or introduce explicit numerical time stepping.
- A practical diagnostic for monitoring temporal convergence.

Limitations

- The method does not guarantee convergence to the true solution.
- Training may still struggle with very long horizons, stiff systems or chaotic dynamics.
- Performance depends on residual scaling, the causal parameter ϵ, and the neural network's approximation capabilities.

Overall: Causal PINNs do not enforce causality through their architecture or a sequential solver. Instead, they embed temporal precedence into the optimization objective, encouraging the network to learn earlier dynamics before later ones.

Bayesian PINNs

Consider a PDE describing a physical system:

$$\frac{\partial u}{\partial t} + \mathcal{N}[u;\lambda] = 0$$

where λ denotes the physical parameters.

A standard PINN approximates the solution through a neural network: $u_\theta(t,x) \approx u(t,x)$.

The network is trained by minimizing a combination of data and physics residuals: $\mathcal{L}(\theta,\lambda) = \mathcal{L}_{\text{data}} + \mathcal{L}_{\text{physics}}$.

However, **real-world observations** are often **sparse, incomplete, and noisy**.

A deterministic PINN returns a single prediction, but does not directly tell us:

- How uncertain is the predicted solution?
- How confident are we about the estimated physical parameters?
- How does measurement noise affect our predictions?

Can we incorporate uncertainty directly into physics-informed learning?

This motivates Bayesian Physics-Informed Neural Networks (B-PINNs).

Bayesian PINNs: From Deterministic to Bayesian PINNs

A deterministic PINN learns one optimal set of network parameters:
$\theta^* = \arg\min_{\theta} \mathcal{L}(\theta)$.

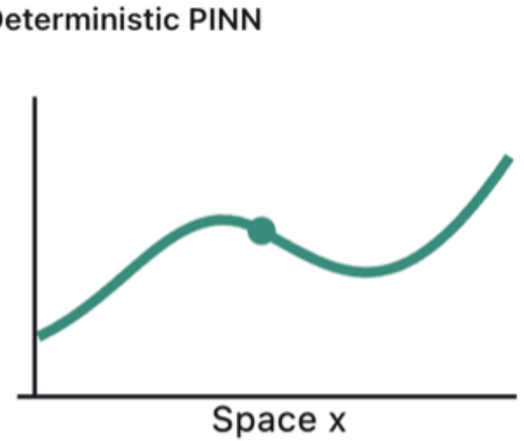
Once trained, it produces a single solution: $u_{\theta^*}(t, x)$.

A **Bayesian PINN** instead treats the **neural network parameters as random variables**: $\theta \sim p(\theta)$.

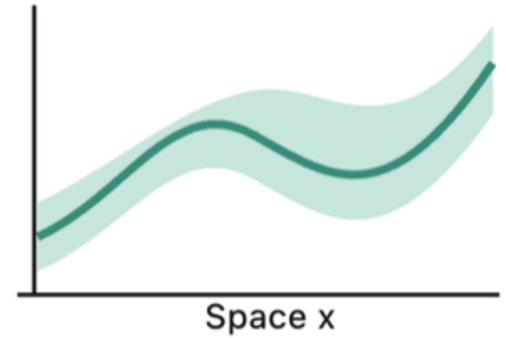
Rather than finding one optimal parameter configuration, we **infer a posterior distribution**: $p(\theta \mid \mathcal{D}, \text{physics})$.

Each posterior sample $\theta^{(k)}$ produces a different candidate solution: $u_{\theta^{(k)}}(t, x)$.

Key idea: Instead of learning a single solution, B-PINNs quantify which solutions remain plausible given both observations and physical knowledge.



One predicted solution.



A distribution of possible solutions.

Bayesian PINNs: Bayesian Inference

Bayes' rule gives: $p(\theta \mid \mathscr{D}) = \dfrac{p(\mathscr{D} \mid \theta)p(\theta)}{p(\mathscr{D})}$

Bayesian inference combines prior knowledge with observed evidence.

where:

- **Prior** $p(\theta)$: Our beliefs about network parameters before observing the data.
- **Likelihood** $p(\mathscr{D} \mid \theta)$: How well a given network explains the observations.
- **Posterior** $p(\theta \mid \mathscr{D})$: Updated distribution after considering the evidence.

A common prior is a Gaussian distribution over network weights: $\theta \sim \mathcal{N}(0, \sigma_\theta^2 I)$.

Unlike standard Bayesian neural networks, **B-PINNs must also incorporate physical knowledge**.

This can be done by **treating the PDE residual as additional probabilistic evidence**:

$$\boxed{p(\theta \mid \mathscr{D}, \mathscr{R} = 0) \propto p(\mathscr{D} \mid \theta)\, p(\mathscr{R} = 0 \mid \theta)\, p(\theta)}.$$

The physical equations therefore contribute to the likelihood, favoring neural networks that satisfy the governing PDE.

Bayesian PINNs: Modeling Noisy Observations

Suppose we observe the physical solution at locations (t_i, x_i): $\mathscr{D} = \{(t_i, x_i, y_i)\}_{i=1}^{N_d}$.

Assume measurements are corrupted by independent Gaussian noise:

$$y_i = u_\theta(t_i, x_i) + \epsilon_i, \qquad \epsilon_i \sim \mathcal{N}(0, \sigma_d^2).$$

The **likelihood** of each observation is: $p(y_i \mid \theta) = \mathrm{N}\left(y_i; u_\theta(t_i, x_i), \sigma_d^2\right)$.

Assuming conditional independence: $p(\mathscr{D} \mid \theta) = \prod_{i=1}^{N_d} \mathrm{N}\left(y_i; u_\theta(t_i, x_i), \sigma_d^2\right)$

Taking the **negative logarithm**, and ignoring terms independent of θ:

$$-\log p(\mathscr{D} \mid \theta) = \frac{1}{2\sigma_d^2} \sum_{i=1}^{N_d} |\, y_i - u_\theta(t_i, x_i) \,|^2 + \text{const}.$$

Note: The familiar mean squared error emerges naturally from a Gaussian measurement-noise assumption. The variance σ_d^2 controls how strongly observations influence the posterior.

Bayesian PINNs: Incorporating Physics into the Likelihood

Key insight: The physics-informed loss becomes a probabilistic constraint on the posterior distribution.

Consider the governing PDE: $u_t + \mathcal{N}[u;\lambda] = 0.$

Define the physics residual: $\mathcal{R}_{\theta,\lambda}(t,x) = \dfrac{\partial u_\theta}{\partial t} + \mathcal{N}[u_\theta;\lambda]\,.$

We evaluate this residual at N_r collocation points: $\{(t_j^r, x_j^r)\}_{j=1}^{N_r}\,.$

Instead of requiring the residual to be exactly zero, we introduce a **Gaussian residual model:**

$$0 = \mathcal{R}_{\theta,\lambda}(t_j^r, x_j^r) + \eta_j, \qquad \eta_j \sim \mathrm{N}(0,\sigma_r^2)\,.$$

This produces a **physics likelihood**: $p(\mathcal{R} = 0 \mid \theta,\lambda) = \prod_{j=1}^{N_r} \mathrm{N}\left(0; \mathcal{R}_{\theta,\lambda}(t_j^r, x_j^r), \sigma_r^2\right)\,.$

Taking the negative logarithm: $$-\log p(\mathcal{R} = 0 \mid \theta,\lambda) = \frac{1}{2\sigma_r^2}\sum_{j=1}^{N_r} \mid \mathcal{R}_{\theta,\lambda}(t_j^r, x_j^r) \mid^2 + \text{const}\,.$$

The parameter σ_r controls the strength of the physics constraint:

- Small σ_r: Strongly favors small PDE residuals.
- Large σ_r: Allows greater deviations from the governing equation.

Bayesian PINNs: Bayesian PINN Posterior

We now combine the three sources of information:

1. Prior knowledge about network weights and PDE parameters.
2. Observed measurements.
3. The governing physical equations.

For an **inverse problem**, both the network parameters θ and the physical parameters λ are unknown.

The posterior becomes:

$$\boxed{\begin{aligned} p(\theta, \lambda \mid \mathscr{D}, \mathscr{R} = 0) \\ \propto p(\mathscr{D} \mid \theta) p(\mathscr{R} = 0 \mid \theta, \lambda) p(\theta) p(\lambda) \,. \end{aligned}}$$

Assuming Gaussian priors and likelihoods, the negative log-posterior is, up to an additive constant:

$$-\log p(\theta, \lambda \mid \mathscr{D}, \mathscr{R} = 0) = \underbrace{\frac{1}{2\sigma_d^2} \sum_{i=1}^{N_d} |\, y_i - u_\theta(t_i, x_i) \,|^2}_{\text{Data fidelity}} + \underbrace{\frac{1}{2\sigma_r^2} \sum_{j=1}^{N_r} |\, \mathscr{R}_{\theta,\lambda}(t_j^r, x_j^r) \,|^2}_{\text{Physics consistency}} + \underbrace{\frac{\|\theta\|^2}{2\sigma_\theta^2} - \log p(\lambda)}_{\text{Prior regularization}} + \text{const} \,.$$

This reveals an important connection with standard PINNs.

Minimizing this objective gives a Maximum A Posteriori (MAP) estimate:

$$\boxed{(\theta_{\text{MAP}}, \lambda_{\text{MAP}}) = \arg\max_{\theta,\lambda} p(\theta, \lambda \mid \mathscr{D}, \mathscr{R} = 0)}.$$

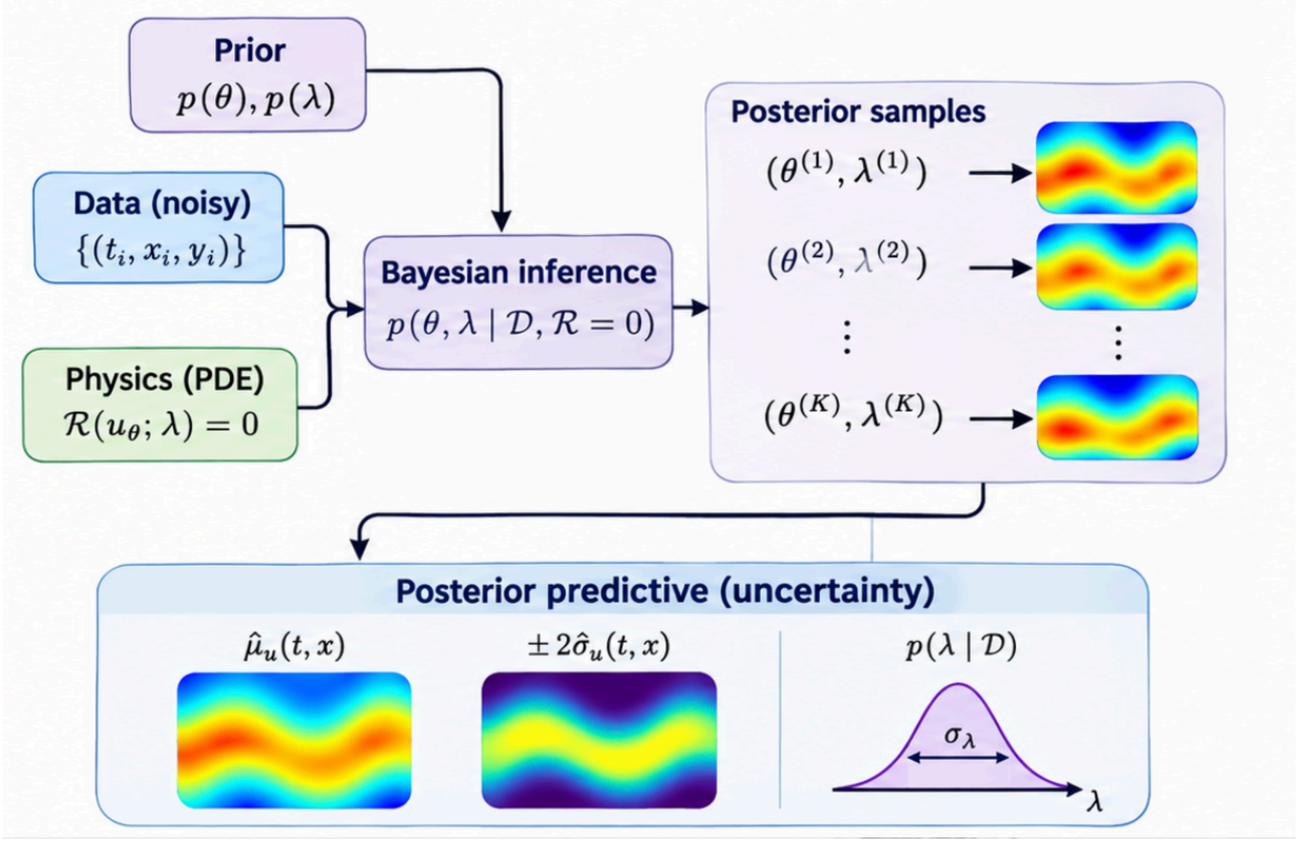
Therefore, a standard PINN with appropriate regularization and loss weights can be interpreted as performing MAP estimation.

But a **Bayesian PINN** goes further: Instead of finding only the posterior mode, it seeks to **characterize the entire posterior distribution**.

Bayesian PINNs: Approximate the Posterior

The **posterior** distribution is generally **intractable** because the neural network is nonlinear and may contain thousands of parameters.

Two major approaches are **Hamiltonian Monte Carlo** (HMC) and **Variational Inference** (VI).



Bayesian PINNs: HMC

HMC generates approximately posterior-distributed parameter samples:

$$(\theta^{(k)}, \lambda^{(k)}) \sim p(\theta, \lambda \mid \mathscr{D}, \mathscr{R} = 0)\,.$$

It uses the gradients of the log-posterior to efficiently explore the parameter space.

A **Hamiltonian** is introduced: $H(\theta, p) = -\log p(\theta \mid \mathscr{D}, \mathscr{R} = 0) + \frac{1}{2} p^\top M^{-1} p$

where p is an auxiliary momentum variable. For inverse problems, λ can be included in the parameter vector.

HMC simulates trajectories in this augmented space and uses a Metropolis correction to preserve the target posterior.

Advantages: Can capture complex posterior dependencies.

Limitations: Computationally expensive because it requires repeated neural-network evaluations and gradient calculations.

Bayesian PINNs: VI

Instead of sampling directly from the true posterior, VI introduces a tractable approximation:

$$q_\phi(\theta, \lambda) \approx p(\theta, \lambda \mid \mathscr{D}, \mathscr{R} = 0)\,.$$

For example, a mean-field Gaussian approximation:

$$q_\phi(\theta) = \prod_{k=1}^{d_\theta} \mathcal{N}(\theta_k; \mu_k, \sigma_k^2)\,.$$

We optimize the variational parameters ϕ by minimizing:

$$D_{\mathrm{KL}}\left(q_\phi \parallel p(\theta, \lambda \mid \mathscr{D}, \mathscr{R} = 0)\right).$$

Advantages: Usually more scalable and compatible with stochastic gradient optimization.

Limitations: The chosen variational family may fail to capture correlations, multiple modes, or the full posterior uncertainty.

Bayesian PINNs: Predictive Uncertainty

Once we have inferred the posterior distribution, we can generate multiple plausible solutions by sampling the network parameters:

$$(\theta^{(k)}, \lambda^{(k)}) \sim p(\theta, \lambda \mid \mathcal{D}, \mathcal{R} = 0)\,.$$

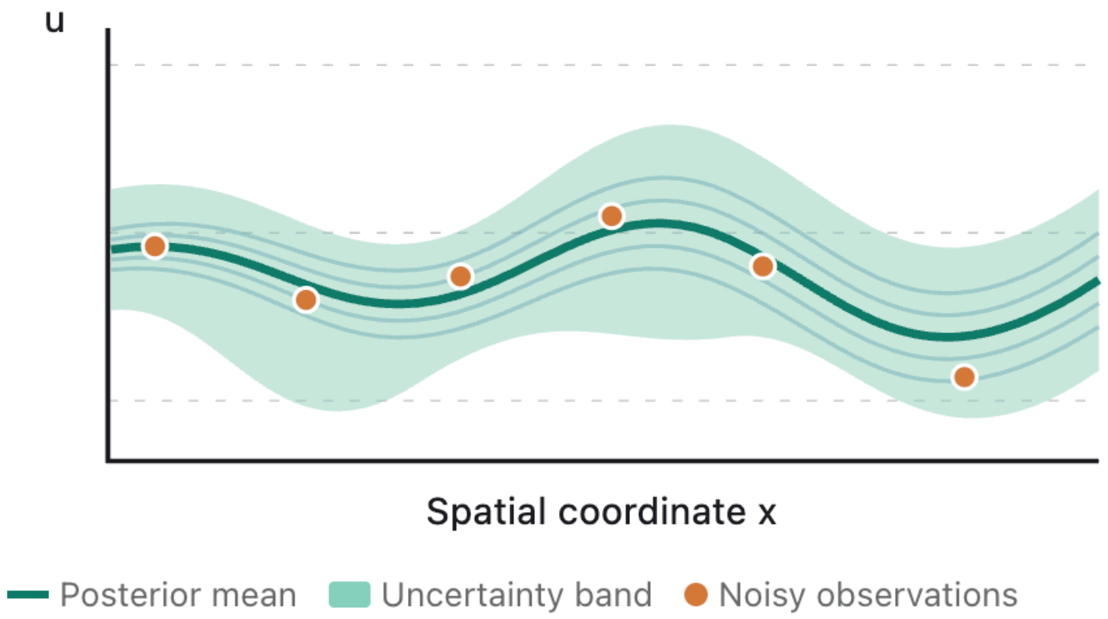
Each sample produces a solution: $u^{(k)}(t,x) = u_{\theta^{(k)}}(t,x)\,.$

The **posterior predictive distribution** is: $p(u_* \mid \mathcal{D}, \mathcal{R} = 0) = \int p(u_* \mid \theta, \lambda) p(\theta, \lambda \mid \mathcal{D}, \mathcal{R} = 0)\, d\theta\, d\lambda\,.$

We can approximate its mean and variance using Monte Carlo samples:

$$\hat{\mu}_u(t,x) = \frac{1}{K} \sum_{k=1}^{K} u^{(k)}(t,x),$$

$$\hat{\sigma}_u^2(t,x) = \frac{1}{K-1} \sum_{k=1}^{K} \left(u^{(k)}(t,x) - \hat{\mu}_u(t,x) \right)^2.$$



Bayesian PINNs: Predictive Uncertainty

Two sources of uncertainty

Epistemic uncertainty: Uncertainty about the solution or physical parameters due to limited information, reflected in the posterior distribution.

Aleatoric uncertainty: Variability arising from measurement noise or intrinsic randomness.

If the observation model is Gaussian with variance σ_d^2, then:

$$\mathrm{Var}(y_* \mid \mathscr{D}) = \underbrace{\mathrm{Var}_\theta(u_\theta(t,x) \mid \mathscr{D})}_{\text{Epistemic}} + \underbrace{\sigma_d^2}_{\text{Aleatoric}} .$$

This decomposition assumes independent observation noise with known, constant variance. Other noise models require the more general law of total variance.

Note: The posterior distribution describes uncertainty about the physical solution, while the predictive distribution of future noisy observations includes additional measurement uncertainty.

Bayesian PINNs

Bayesian PINNs are especially attractive for inverse problems with sparse and noisy observations.

Example: Identifying an Unknown Diffusion Coefficient

Consider: $u_t = \kappa u_{xx}, \qquad \kappa > 0.$

We observe noisy temperature measurements: $y_i = u(t_i, x_i) + \epsilon_i$.

A deterministic PINN estimates a single diffusion coefficient: κ^* .

A Bayesian PINN instead estimates: $p(\kappa \mid \mathscr{D}, \mathscr{R} = 0)$.

This allows us to quantify uncertainty in the estimated physical parameter and propagate that uncertainty to future temperature predictions.

Bayesian PINNs

	Standard PINNs	Bayesian PINNs
Network parameters	Deterministic	Probability distribution
Training	Loss minimization	Posterior inference
Solution	Point estimate	Predictive distribution
PDE parameters	Point estimates	Posterior distributions
Noisy data	Data-fitting loss	Explicit noise likelihood
Computational cost	Relatively lower	Often substantially higher

Bayesian PINNs

Main Advantages

- Quantify uncertainty in both solutions and unknown physical parameters.
- Naturally incorporate noisy and incomplete observations.
- Combine prior knowledge, physical equations and data within one probabilistic framework.
- Support uncertainty-aware predictions and scientific inference.

Main Limitations

- Computational cost: Posterior inference can require many neural-network evaluations.
- Approximation quality: Variational approximations may underestimate uncertainty or miss posterior modes.
- Prior sensitivity: Results can depend strongly on priors and assumed noise distributions.
- Physics misspecification: If the governing PDE is inaccurate, strong physical constraints may produce misleading posterior distributions.
- No automatic calibration guarantee: Bayesian inference does not ensure reliable uncertainty estimates when the model or posterior approximation is inadequate.

Overall: Bayesian PINNs extend physics-informed learning from point estimation to probabilistic inference, enabling joint estimation of physical solutions, unknown PDE parameters, and their associated uncertainties.

Bastek, Sun, Kochmann. "Physics-informed diffusion models." ICLR, 2025

Physics-Informed Diffusion Models

Consider a physical system governed by a PDE: $\frac{\partial u}{\partial t} + \mathcal{N}[u] = 0$

We want to **generate multiple physically plausible solutions** rather than learning a single deterministic approximation.

Traditional **diffusion models** can learn the distribution of physical solutions from simulation data:

$$u \sim p_{\text{data}}(u)\,.$$

However, **generated samples may violate physical constraints**, such as:

- Conservation of mass or energy.
- Governing differential equations.
- Initial and boundary conditions.
- Structural or geometric constraints.

Can we combine the generative capabilities of diffusion models with the physical consistency of PINNs?

The objective is to learn a distribution of solutions that are both realistic and physically admissible.

Physics-Informed Diffusion Models

Recall that a standard **PINN** approximates a solution: $u_\theta(t,x) \approx u(t,x)$

and minimizes a physics-informed loss: $\mathscr{L}_{\text{PINN}} = \mathscr{L}_{\text{data}} + \lambda \mathscr{L}_{\text{PDE}}$

Its objective is to **find one solution consistent with the PDE** and the available observations.

A **diffusion model** instead learns to **generate different samples** from a distribution:

$$z \sim \mathcal{N}(0,I) \longrightarrow u^{(1)}, u^{(2)}, \ldots$$

A **Physics-Informed Diffusion Model** combines these objectives:

$$\underbrace{\text{Learn the data distribution}}_{\text{Generative modeling}} + \underbrace{\text{Enforce physical constraints}}_{\text{Physics-informed learning}}$$

Note: Instead of constraining one predicted solution, we want the physical requirements to hold across the distribution of generated samples.

Denoising Diffusion

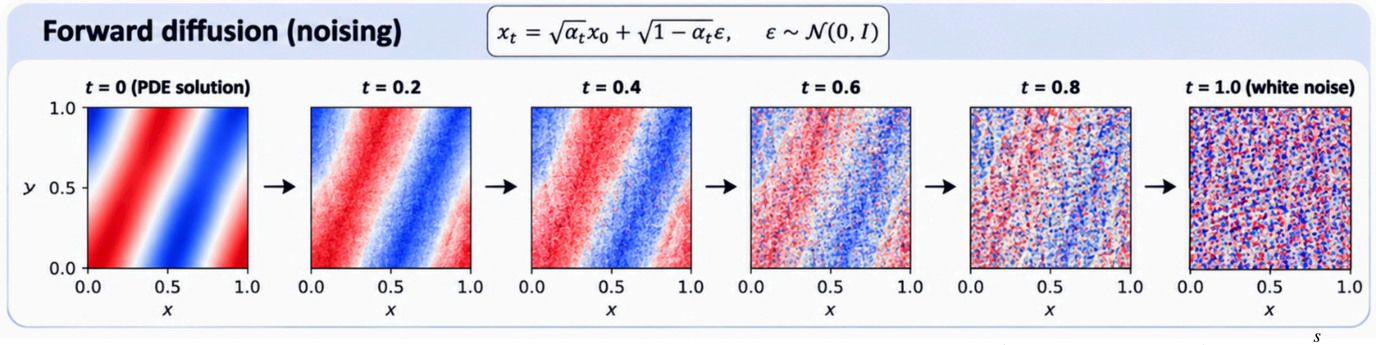
Important: The diffusion index s is an artificial generative time, distinct from the physical time t appearing in the PDE!!

Let u_0 represent a physical solution, such as a temperature field, velocity field, or complete spatio-temporal trajectory.

A diffusion model defines a **forward noising process**: $u_s = \sqrt{\bar{\alpha}_s} u_0 + \sqrt{1 - \bar{\alpha}_s}\epsilon$

where:

- $s \in \{1, \ldots, S\}$ is the diffusion step.
- $\epsilon \sim \mathcal{N}(0, I)$ is Gaussian noise.
- $\bar{\alpha}_s$ controls the noise level.



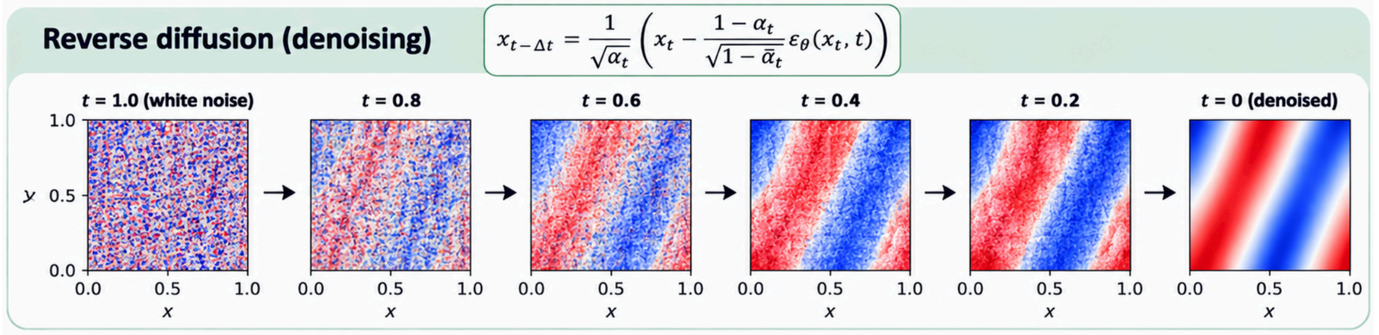
$$q(u_s \mid u_{s-1}) = = \mathcal{N}\left(\sqrt{\alpha_t}\, u_{s-1}, (1 - \alpha_s) I\right) \qquad \bar{\alpha}_s = \prod_{t=1}^{s} \alpha_t$$

$$u_s = \sqrt{\alpha_s}\, u_{s-1} + \sqrt{1 - \alpha_s}\, \epsilon_s, \qquad \epsilon_s \sim \mathcal{N}(0, I)\,.$$

The neural network is trained to predict the noise: $\epsilon_\theta(u_s, s) \approx \epsilon$.

The standard denoising objective is: $\mathcal{L}_{\text{diffusion}} = \mathbb{E}_{u_0, s, \epsilon}\left[\|\epsilon - \epsilon_\theta(u_s, s)\|^2\right]$

During **generation**, the process is reversed:



Denoising Diffusion

The Score: A Direction Toward High Probability

Consider a probability density $p(x)$.

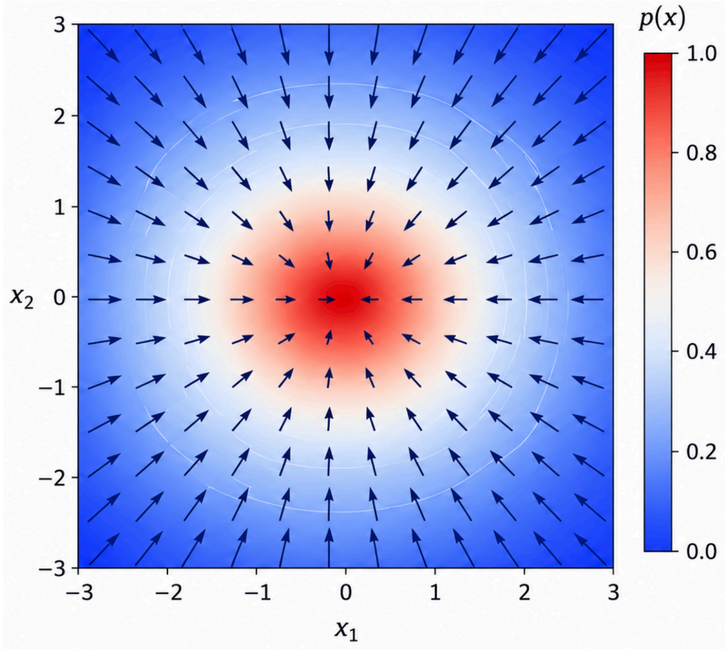
Its score function is defined as: $s(x) = \nabla_x \log p(x)$.

The score is a **vector field** indicating the direction in which the log-probability density increases most rapidly.

For example, for a Gaussian distribution: $p(x) = \mathrm{N}(\mu, \sigma^2 I)$,

the score is: $s(x) = -\dfrac{x - \mu}{\sigma^2}$.

Therefore, the score points toward the Gaussian mean.



Score of a Gaussian distribution

However, following the score through gradient ascent does not generate samples from the distribution: it tends to concentrate points around its modes.

To obtain sampling, we need to combine the score with an appropriate stochastic dynamics.

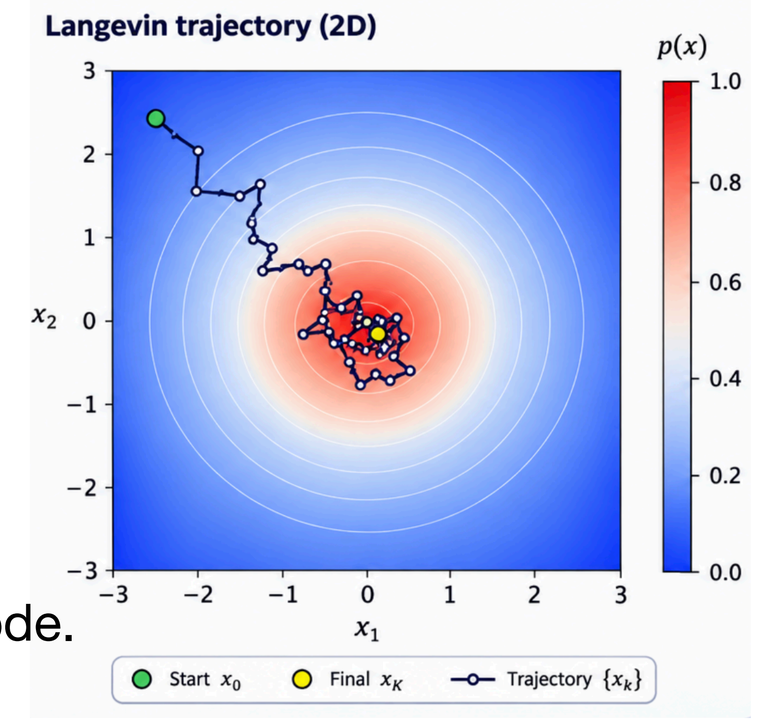
Denoising Diffusion

From the Score to Sampling

One way to sample from a distribution using its score is through Langevin dynamics: $x_{k+1} = x_k + \eta s(x_k) + \sqrt{2\eta}\,\xi_k, \qquad \xi_k \sim \mathrm{N}(0,I)\,.$

The two terms have complementary roles:

- **Score:** Drives samples toward regions of higher probability.
- **Noise:** Allows exploration and prevents deterministic convergence to a mode.



Under suitable assumptions, the corresponding continuous-time Langevin process has $p(x)$ as its stationary distribution.

Diffusion models exploit a related idea, but instead of using a single distribution, they **learn the scores of progressively noisier distributions**:

$$s_\theta(x_s, s) \approx \nabla_{x_s} \log p_s(x_s)\,.$$

The generation process starts from an approximately Gaussian distribution p_S and progressively recovers $p_0 = p_{\text{data}}$.

The score provides the information needed to reverse the diffusion process.

Denoising Diffusion

$$q(x_s \mid x_0) = \mathcal{N}\left(\sqrt{\bar{\alpha}_s} x_o, (1 - \bar{\alpha}_s) I\right)$$
$$s(x_s, s \mid x_0) = \nabla_{x_s} \log q(x_s \mid x_0)$$

Connection Between Noise Prediction and Score Estimation

We usually train a DDPM to predict noise rather than explicitly predict the score.

Recall: $x_s = \sqrt{\bar{\alpha}_s} x_0 + \sqrt{1 - \bar{\alpha}_s} \epsilon$.

For this Gaussian corruption process, the **noise predictor is related to the score** by:

$$s_\theta(x_s, s) \approx -\frac{\epsilon_\theta(x_s, s)}{\sqrt{1 - \bar{\alpha}_s}} .$$

Therefore, a network trained to predict the added noise implicitly learns the score of the noisy data distribution!

The reverse sampling procedure can use either the estimated score or the equivalent noise prediction.

For example, in a DDPM: $\mu_\theta(x_s, s) = \frac{1}{\sqrt{\alpha_s}} \left(x_s - \frac{1 - \alpha_s}{\sqrt{1 - \bar{\alpha}_s}} \epsilon_\theta(x_s, s) \right)$.

We then sample: $x_{s-1} = \mu_\theta(x_s, s) + \sigma_s \xi, \qquad \xi \sim \mathrm{N}(0, I)$.

This is the standard reverse diffusion process.

$p_\theta(x_{s-1} \mid x_s)$

Forward: $x_s = \sqrt{\alpha_s} x_{s-1} + \sqrt{1 - \alpha_s} \epsilon_s$.

Reverse: $x_{s-1} = \frac{1}{\sqrt{\alpha_s}} \left(x_s - \sqrt{1 - \alpha_s} \epsilon_s \right)$

Diffusion Models and PDEs

Suppose we train a diffusion model to generate solutions of the heat equation: $u_t - \kappa u_{xx} = 0$.

A generated sample $\hat{u}(t, x)$ may resemble the training data while violating the PDE.

Define its physical residual: $\mathscr{R}(\hat{u}) = \dfrac{\partial \hat{u}}{\partial t} - \kappa \dfrac{\partial^2 \hat{u}}{\partial x^2}$.

For an exact solution: $\mathscr{R}(\hat{u}) = 0$.

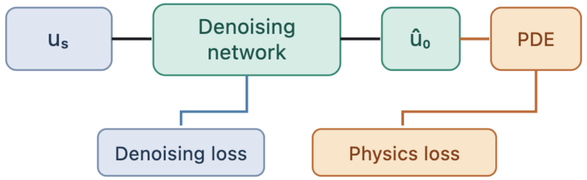
For a generated approximation, the residual is generally nonzero.

A possible physics loss is: $\mathscr{L}_{\text{physics}} = \dfrac{1}{N_c} \displaystyle\sum_{i=1}^{N_c} | \mathscr{R}(\hat{u})(t_i, x_i) |^2$.

Here, the residual is evaluated at physical collocation points or using a suitable numerical discretization.

The problem: Standard diffusion training encourages agreement with the data distribution, but does not explicitly penalize violations of the governing PDE.

Physics-Informed Diffusion Models



The simplest approach is to **incorporate a physics-based penalty into the diffusion training objective.**

Given a noisy sample u_s, the denoising network produces a prediction of the clean solution:

$$\hat{u}_0(u_s, s) = \frac{u_s - \sqrt{1-\bar{\alpha}_s}\,\epsilon_\theta(u_s, s)}{\sqrt{\bar{\alpha}_s}}\,.$$

$\hat{u}_0 = \mathbb{E}[u_0 \mid u_s]$
expected solution

We can then **evaluate the physical residual on this reconstructed solution:** $\mathscr{R}(\hat{u}_0)$.

The training objective becomes: $\mathscr{L}_{\text{PIDM}} = \mathscr{L}_{\text{diffusion}} + \lambda_{\text{phys}}\mathscr{L}_{\text{physics}}$

with a physics penalty:

$$\mathscr{L}_{\text{physics}} = \mathbb{E}_{u_0,s,\epsilon}\left[\|\mathscr{R}(\hat{u}_0(u_s, s))\|^2\right].$$

Key idea: Physics is incorporated during training, so the denoising network learns to generate samples that better satisfy the governing equations.

Physics-Informed Diffusion Models

An important subtlety is that the noisy state u_s is not itself expected to satisfy the PDE.

Indeed: $u_s = \sqrt{\bar{\alpha}_s} u_0 + \sqrt{1 - \bar{\alpha}_s} \epsilon$.

Even if u_0 is a valid physical solution, adding Gaussian noise will generally destroy its physical consistency.

Therefore, imposing $\mathscr{R}(u_s) = 0$ would be inappropriate.

$\hat{u}_0$ expected solution

Instead, we **enforce physics on a reconstructed clean sample:** $\mathscr{R}(\hat{u}_0(u_s, s)) \approx 0.$

There is an additional choice: should we evaluate the (i) residual on the predicted mean, or on a (ii) sample drawn from the reverse distribution?

For nonlinear differential operators, these are not equivalent: $\mathscr{R}(\mathbb{E}[u]) \neq \mathbb{E}[\mathscr{R}(u)]$ in general.

A physically valid mean does not guarantee physically valid individual samples and viceversa.

Bastek et al. therefore **distinguish between denoised predictions** and **physics-informed sampling strategies**, allowing physical constraints to act on generated realizations rather than only their mean.

This distinction is particularly important when the physical constraints are nonlinear.

Physics-Informed Diffusion Models

Why Can the Denoised Mean Violate Physics?

Consider a simple physical constraint: $\mathscr{R}(x) = x_1^2 + x_2^2 - 1 = 0.$

This requires all valid solutions to lie on the unit circle.

Suppose our training dataset contains samples distributed around this circle:

$$x_0 \sim p_{\text{data}}(x_0), \qquad \|x_0\| = 1.$$

All training samples satisfy the constraint exactly.

However, after adding substantial Gaussian noise, the denoising network may be uncertain about which point on the circle generated the noisy observation.

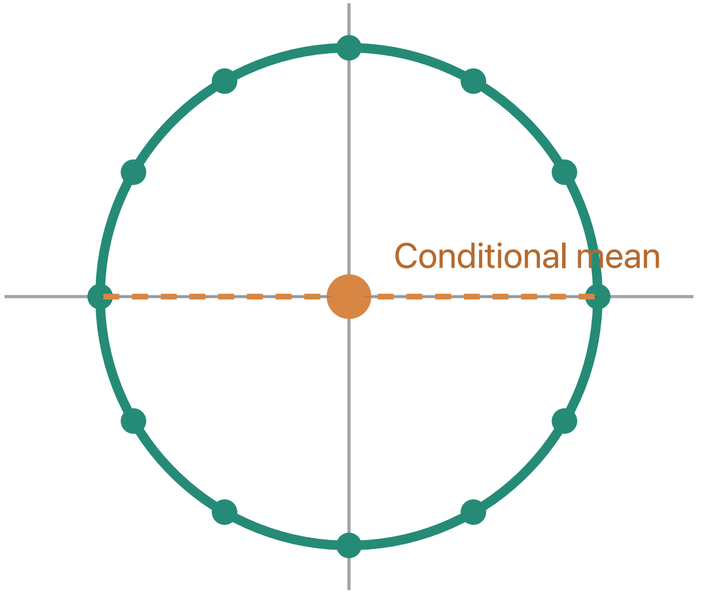
For a symmetric distribution of possible clean solutions:

$$\hat{x}_0 = \mathbb{E}[x_0 \mid x_s] = (0,0)\,.$$

But: $\mathscr{R}(\hat{x}_0) = 0^2 + 0^2 - 1 = -1.$

The conditional mean violates the constraint, even though the individual solutions are valid.

This is the key difficulty when applying physical constraints to denoised predictions.



All green points satisfy the constraint

Solution 1: Mean Estimation with Noise-Dependent Physics Weighting

The first strategy is **Mean Estimation (PIDM-ME)**.

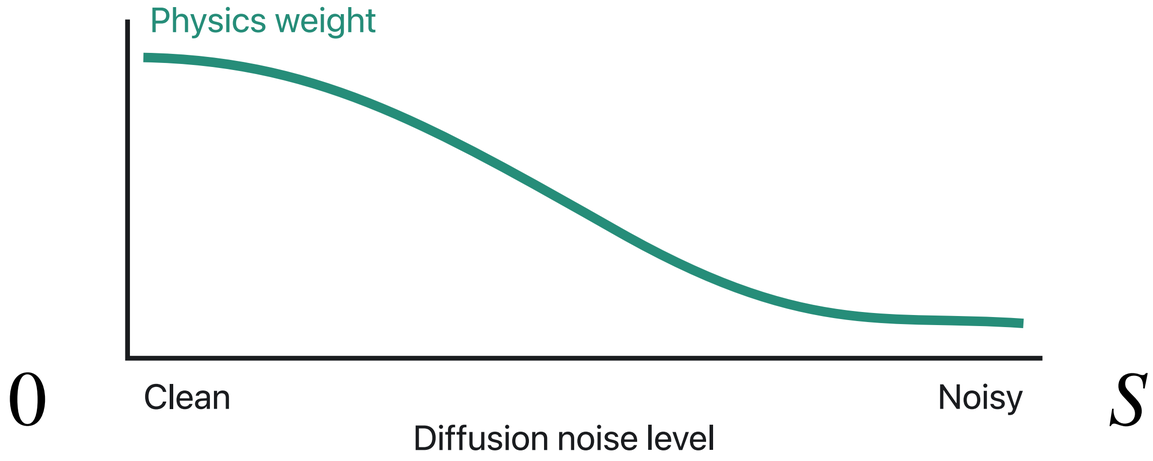
We evaluate the physics residual on the denoiser's estimate of the clean solution: $x_0^* = \hat{x}_0(x_s, s)$.

However, we do not enforce the physics constraint equally at all diffusion timesteps.

Instead, we introduce a noise-dependent physics weight: $\mathscr{L}^{(s)}_{\text{physics}} = w_s \| \mathscr{R}(\hat{x}_0) \|^2$.

The intuition:

- At high noise levels, $\hat{x}_0$ may average multiple valid solutions. We therefore apply a weaker physics penalty.
- At low noise levels, the clean solution is less ambiguous. We can enforce physics more strongly.



Solution 1: Mean Estimation with Noise-Dependent Physics Weighting

Probabilistic formulation

Treat physical consistency as a virtual observation: $\hat{r} = 0$.

Assume: $q_{\mathscr{R}}(0 \mid x_0^*) = \mathrm{N}\left(0; \mathscr{R}(x_0^*), \frac{\Sigma_s}{c} I\right)$.

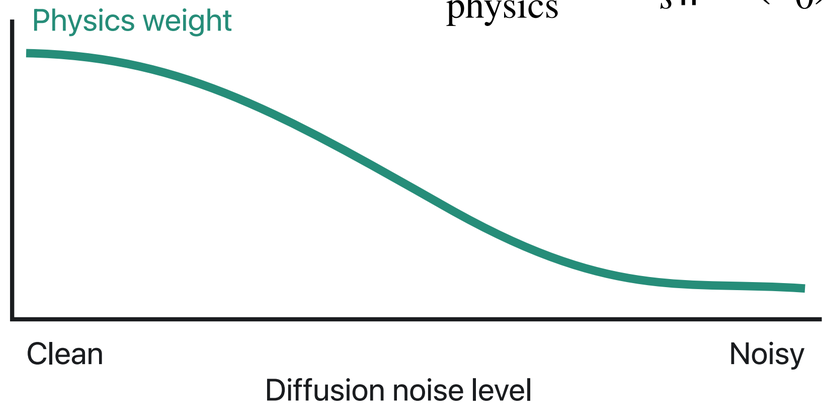
The higher Σ_s the more tolerant we are wrt non zero residuals

The corresponding negative log-likelihood gives: $w_s = \frac{c}{2\Sigma_s}$.

$-\log q_{\mathscr{R}}(0 \mid x_0^*) = w_s \|\mathscr{R}(\hat{x}_0)\|^2.$

Therefore: $\boxed{\mathscr{L}^{(s)}_{\text{physics}} = \frac{c}{2\Sigma_s}\|\mathscr{R}(\hat{x}_0)\|^2}$.

$\mathscr{L}^{(s)}_{\text{physics}} = w_s \|\mathscr{R}(\hat{x}_0)\|^2.$



Here, $\boxed{\Sigma_s}$ is a prescribed **reverse-process variance** and $\boxed{c}$ controls the overall strength of physics enforcement. The **variance schedule** is chosen to relax the physical constraint for noisier reconstructions, with special handling of the final deterministic step.

Advantage: No additional denoising steps are required.

Limitation: The residual is still evaluated on a conditional mean, so the underlying mean-versus-sample conflict is not eliminated, only mitigated.

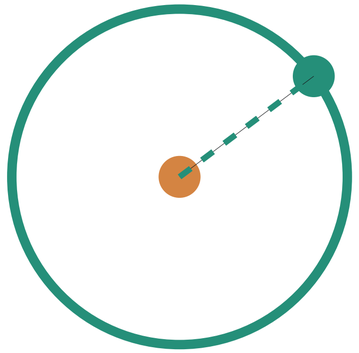
Solution 2: Evaluate Physics on a Generated Sample

The second strategy is **Sample Estimation (PIDM-SE).**

Rather than evaluating physics directly on the conditional mean, we obtain an approximate clean realization using **accelerated reverse diffusion**.

Starting from x_s: $x_s \xrightarrow{\text{DDIM}} x_1 \xrightarrow{\text{denoising}} x_0^*$.

We then evaluate: $\mathscr{L}_{\text{physics}}^{(s)} = \frac{c}{2\Sigma_s}\|\mathscr{R}(x_0^*)\|^2.$



Valid samples on the circle

Sample estimation

Approximate reverse diffusion

→ x_0^*

Why does this help?

Return to the unit-circle example.

Under high uncertainty, the conditional mean may be: $\hat{x}_0 = (0,0)$.

However, an individual clean sample might instead be: $x_0^* = (0.8,0.6)$.

This sample satisfies: $\mathscr{R}(x_0^*) = 0.8^2 + 0.6^2 - 1 = 0.$

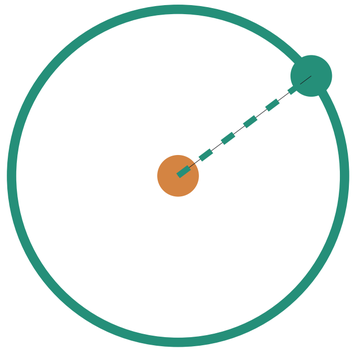
The distinction is that the physics constraint now acts on an individual generated realization rather than only on the estimated mean

Solution 2: Evaluate Physics on a Generated Sample

A full reverse diffusion chain at every training iteration would be expensive. The paper therefore uses a two-step accelerated DDIM-based estimate, which reduces computational overhead.

Advantage: Better aligns the physics penalty with individual generated solutions.

Limitation: More expensive than mean estimation, and the accelerated estimate is only an approximation to a complete generated sample.



Valid samples on the circle

Sample estimation

Approximate reverse diffusion

$\rightarrow x_0^*$

Putting Everything Together: **The Physics-Informed Loss**

Both strategies can be expressed using a common estimated clean sample x_0^*:

$$x_0^* = \begin{cases} \hat{x}_0(x_s, s), & \text{Mean Estimation,} \\ \mathrm{DDIM}(x_s \to x_0), & \text{Sample Estimation .} \end{cases}$$

The complete training objective becomes:

$$\mathscr{L}_{\mathrm{PIDM}} = \mathbb{E}_{x_0,s,\epsilon}\Big[\underbrace{\lambda_s \|x_0 - \hat{x}_0(x_s, s)\|^2}_{\text{Learn the data distribution}} + \underbrace{\frac{c}{2\Sigma_s}\|\mathscr{R}(x_0^*)\|^2}_{\text{Encourage physical consistency}}\Big].$$

The two terms have complementary objectives.

Denoising loss: Preserve the statistical diversity and structure of the physical solutions.

Physics loss: Encourage generated solutions to satisfy governing equations and other constraints.

An additional important observation is that physics alone does not determine the distribution of solutions.

In our unit-circle example, minimizing only: $\mathscr{R}(x)^2 = (x_1^2 + x_2^2 - 1)^2$ would allow the generator to produce the same point $(1,0)$ every time.

This would satisfy physics perfectly, but completely miss the desired distribution around the circle.

Therefore: Physical validity $\neq$ Distribution matching .

The physics penalty complements the data objective rather than replacing it.

Guided Generation

Conditioning Through Bayes' Rule

Suppose we want to generate samples satisfying a condition c.

Instead of sampling from $p(x)$, we want: $p(x \mid c) \propto p(x)p(c \mid x)$.

Taking the gradient of the log-density:

$$\nabla_x \log p(x \mid c) = \underbrace{\nabla_x \log p(x)}_{\text{Data score}} + \underbrace{\nabla_x \log p(c \mid x)}_{\text{Guidance score}} .$$

This gives a fundamental interpretation:

- **Data score:** Encourages samples to remain plausible under the learned distribution.
- **Guidance score:** Encourages samples to satisfy the desired condition.

For a diffusion model, the same identity applies at each noise level:

$$\nabla_{x_s} \log p_s(x_s \mid c) = s_\theta(x_s, s) + \nabla_{x_s} \log p_s(c \mid x_s) .$$

Key idea: Conditional generation can be achieved by modifying the score used during reverse diffusion.

The identity is exact when the true unconditional and conditional scores are available. Practical guidance methods approximate one or both terms.

Physics-Guided Sampling

From Physical Constraints to Guidance

Consider a physical system governed by a PDE: $\frac{\partial u}{\partial t} + \mathcal{N}[u] = 0.$

We want to generate solution fields u satisfying the governing equation.

Define a physics-based energy:

$$E_{\text{phys}}(u) = \frac{1}{N_c} \sum_{i=1}^{N_c} \left| \frac{\partial u}{\partial t}(t_i, x_i) + \mathcal{N}[u](t_i, x_i) \right|^2 .$$

Physical consistency corresponds to: $E_{\text{phys}}(u) \approx 0.$

We introduce an artificial constraint likelihood: $p(c \mid u) \propto \exp(-\lambda E_{\text{phys}}(u))\,.$

Therefore, the clean-space **guidance direction** is:

$$\nabla_u \log p(c \mid u) = -\lambda \nabla_u E_{\text{phys}}(u)\,.$$

Interpretation: The negative energy gradient points toward configurations with smaller physical violations.

However, diffusion operates on noisy variables x_s, not directly on the clean solution $u = x_0$.

Question: How can we use a physical constraint defined on clean solutions to guide noisy diffusion states?

Physics-Guided Sampling

Physics Guidance Through the Predicted Clean Sample

At diffusion step s, we have a noisy state x_s.

The pretrained denoising network predicts the noise $\epsilon_\theta(x_s, s)$, allowing us to reconstruct an estimate of the clean sample: $\hat{x}_0(x_s, s) = \dfrac{x_s - \sqrt{1-\bar{\alpha}_s}\,\epsilon_\theta(x_s, s)}{\sqrt{\bar{\alpha}_s}}$.

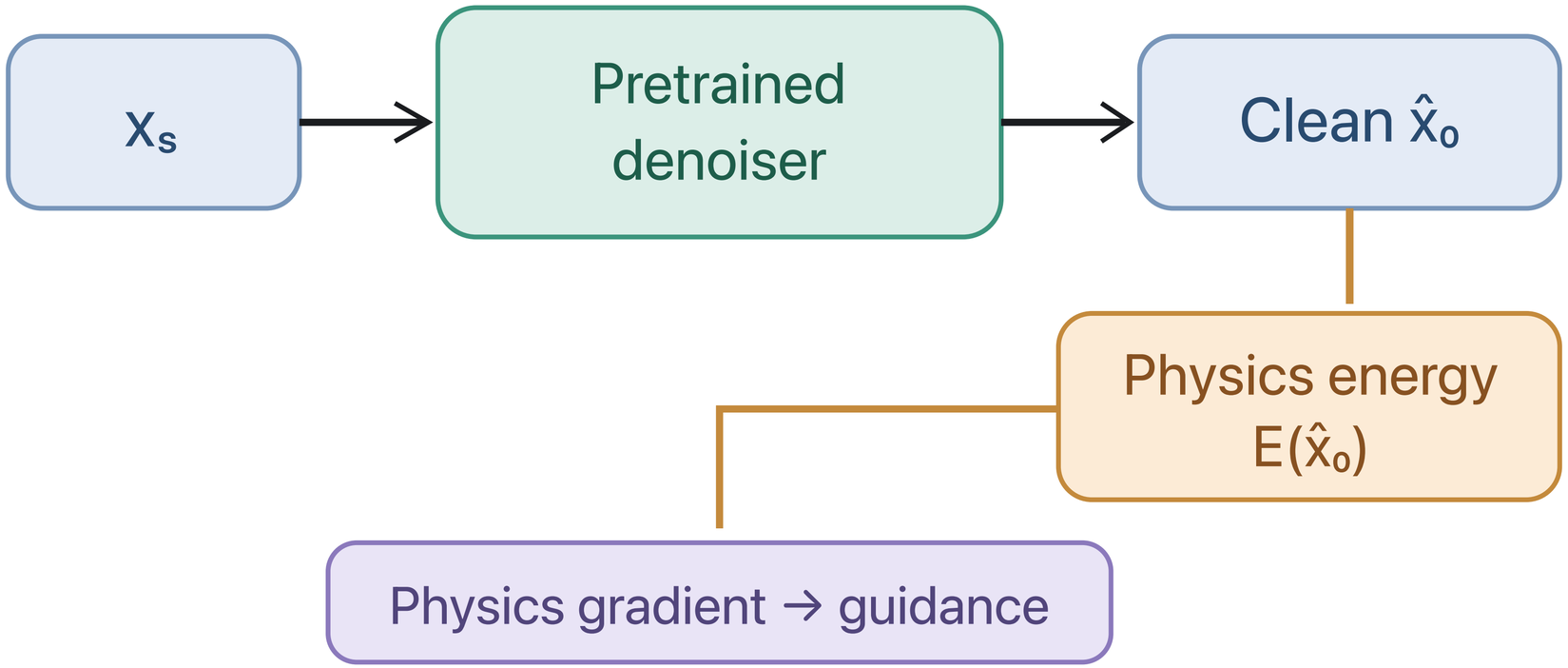
We then evaluate the physical constraint on this clean estimate: $E_{\text{phys}}(\hat{x}_0)$.

The approximate guided score becomes: $s_{\text{guided}}(x_s, s) = s_\theta(x_s, s) - \gamma_s \nabla_{x_s} E_{\text{phys}}(\hat{x}_0(x_s, s))$.

The gradient is calculated using the chain rule:

$$\nabla_{x_s} E_{\text{phys}}(\hat{x}_0) = \left(\frac{\partial \hat{x}_0}{\partial x_s}\right)^{\top} \nabla_{\hat{x}_0} E_{\text{phys}}(\hat{x}_0)\,.$$

Physics-Guided Sampling



Important: This is an approximation to the true conditional guidance score. It is particularly useful because it works with differentiable constraints that were not included during model training.

Physics-Guided Sampling

Consider a heat-equation solution with a small perturbation violating the PDE.

Reverse diffusion

At each diffusion step:

1. **Denoising:** The network estimates a cleaner solution $\hat{u}_0$.
2. **Physics evaluation:** Compute $\mathscr{R}(\hat{u}_0)$.
3. **Gradient computation:** Differentiate the physics loss with respect to the current noisy state.
4. **Guided update:** Combine the denoising direction with the physics correction.
5. **Repeat:** Continue until a final sample is generated.

The physics gradient is obtained through the reconstruction $\hat{u}_0(u_s, s)$, so the chain rule gives:

$$\nabla_{u_s} E_{\text{phys}}(\hat{u}_0) = \left(\frac{\partial \hat{u}_0}{\partial u_s} \right)^{\top} \nabla_{\hat{u}_0} E_{\text{phys}}(\hat{u}_0) \,.$$

This can involve differentiating through the denoising network and the PDE residual.

Trade-off: Physics-guided sampling offers flexibility **without retraining** the model, but may require **expensive gradient computations** at every diffusion step.

Physics-Informed Training vs. Guided Sampling

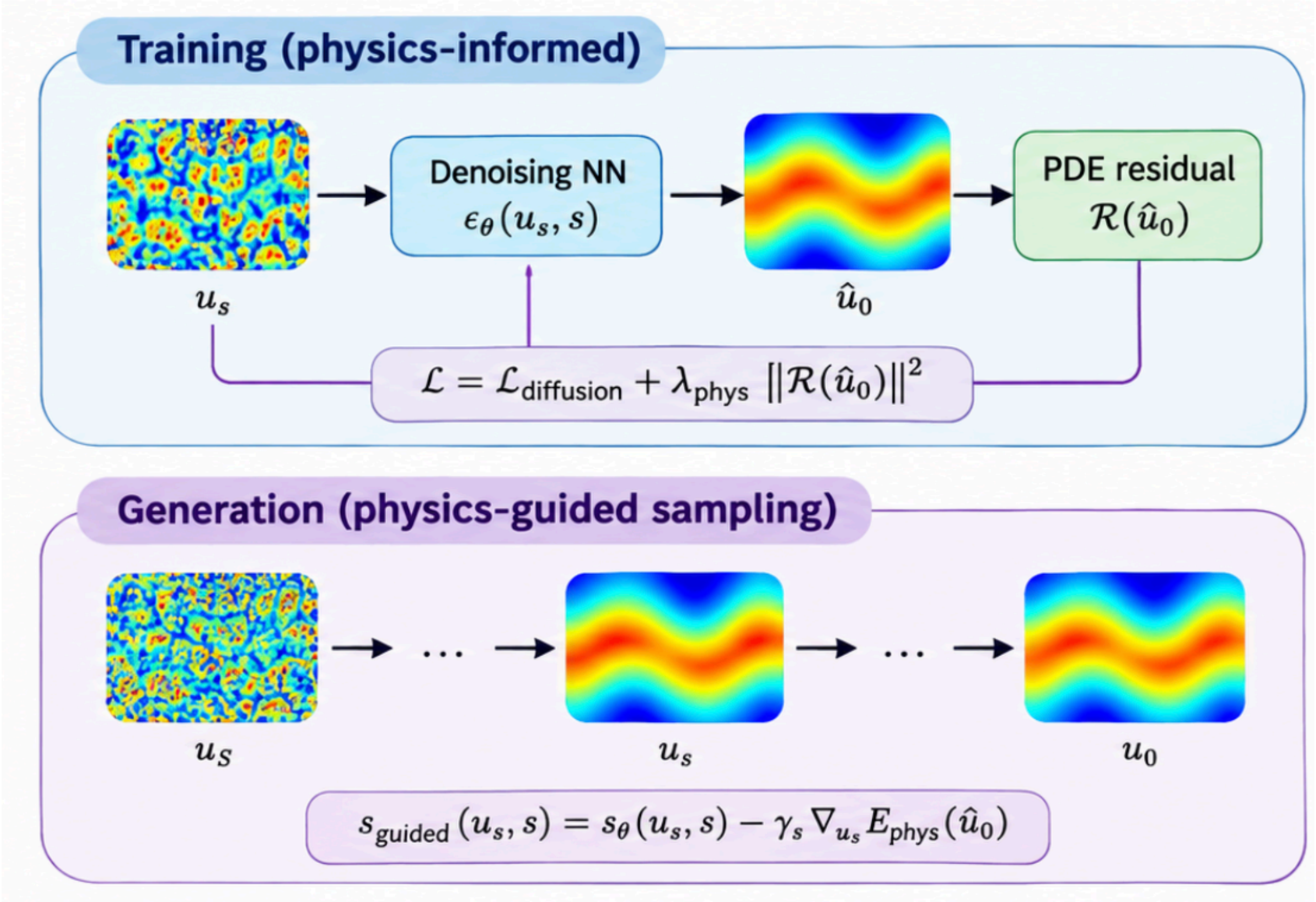
The two strategies are not mutually exclusive.

For example, we could train a diffusion model using a physics-informed loss and then apply additional guidance at inference time to satisfy new observations or design constraints.

Important: Neither approach guarantees exact satisfaction of the PDE in general. Their effectiveness depends on the physics objective, optimization, guidance strength, numerical discretization, and model accuracy.

	Physics-Informed Training	**Physics-Guided Sampling**
When physics is enforced	During model training	During generation
Training objective	Denoising + physics loss	Standard denoising
Sampling	Usually standard reverse diffusion	Modified reverse diffusion
New physical constraints	May require retraining	Can often change guidance
Computational overhead	Mainly during training	During every sampling step
Physical consistency	Encouraged through learned distribution	Encouraged through sample-specific corrections

Physics-Informed Training vs. Guided Sampling



Material

PINA Library (PINNs and NOs): https://pina-org.github.io/PINA/

Tutorial on PINA by Dario Coscia: Tutorial on PINA by Dario Coscia-20251202_111359-Meeting Recording.mp4

Coming next…

Uncertainty quantification